

学校代码: 10246  
学 号: 22307130047

復旦大學

本 科 毕 业 论 文

面向 AIGC 的扩散式无限制对抗样本  
生成与评估方法研究

Diffusion-based Unrestricted Adversarial Example  
Generation and Evaluation for AIGC

院 系: 计算与智能创新学院  
专 业: 保密技术  
姓 名: 蒋超逸  
指 导 教 师: 赵卫东  
完 成 日 期: 2026 年 5 月



# 目 录

|                         |     |
|-------------------------|-----|
| 摘要                      | ii  |
| Abstract                | iii |
| 第 1 章 绪论                | 1   |
| 1.1 研究背景与意义             | 1   |
| 1.2 研究现状与问题             | 2   |
| 1.2.1 对抗攻击研究现状          | 2   |
| 1.2.2 对抗防御研究现状          | 2   |
| 1.2.3 鲁棒性评估工具研究现状       | 3   |
| 1.3 研究目标与内容             | 3   |
| 1.4 论文组织结构              | 4   |
| 第 2 章 相关工作              | 5   |
| 2.1 传统对抗攻击与防御           | 5   |
| 2.2 无限制对抗样本与生成式攻击       | 5   |
| 2.2.1 基于 GAN 的生成式方法     | 7   |
| 2.2.2 基于扩散模型的生成式方法      | 7   |
| 2.2.3 扩散反演与多模态前沿路线      | 8   |
| 2.3 AIGC 安全检测与防御        | 9   |
| 2.3.1 扩散净化方法            | 9   |
| 2.3.2 轻量级防御方法           | 9   |
| 2.4 鲁棒性评估工具与实验协议        | 10  |
| 第 3 章 扩散式无限制对抗样本生成与评估方法 | 11  |
| 3.1 问题定义                | 11  |
| 3.1.1 候选生成算子的形式化定义      | 11  |
| 3.2 候选搜索算法              | 13  |
| 3.2.1 成功判定              | 13  |
| 3.2.2 候选选择准则            | 13  |

|              |                     |           |
|--------------|---------------------|-----------|
| 3.2.3        | 早停与失败回退             | 14        |
| 3.2.4        | 查询复杂度分析             | 15        |
| 3.3          | 多维评估协议              | 16        |
| 3.3.1        | 防御有效性判定             | 17        |
| 3.3.2        | 多种子聚合协议             | 18        |
| 3.4          | 方法扩展性               | 19        |
| <b>第 4 章</b> | <b>算法实现与实验支撑系统</b>  | <b>20</b> |
| 4.1          | 技术选型                | 20        |
| 4.2          | 模型库实现               | 21        |
| 4.3          | 对抗样本生成模块            | 21        |
| 4.3.1        | 单步梯度扰动生成            | 21        |
| 4.3.2        | 迭代投影梯度优化            | 21        |
| 4.3.3        | 生成器基线方法             | 22        |
| 4.3.4        | 扩散式候选搜索生成           | 22        |
| 4.4          | 输入变换与净化模块           | 23        |
| 4.4.1        | 轻量级输入变换方法           | 23        |
| 4.4.2        | 扩散式输入净化方法           | 23        |
| 4.5          | 攻防评估流水线             | 24        |
| 4.6          | 指标计算模块              | 25        |
| 4.7          | 配置化运行机制             | 26        |
| <b>第 5 章</b> | <b>实验与分析</b>        | <b>28</b> |
| 5.1          | 实验设置                | 28        |
| 5.1.1        | 代码与配置组织             | 28        |
| 5.1.2        | 评价指标定义              | 29        |
| 5.1.3        | 同类算法比较口径            | 29        |
| 5.2          | 传统扰动方法基线分析          | 30        |
| 5.3          | DIFFSEARCH 候选搜索效果分析 | 30        |
| 5.4          | 与同类攻击方法的比较          | 31        |
| 5.5          | 攻防交互与鲁棒性矩阵分析        | 32        |
| 5.5.1        | 防御代价分析              | 34        |
| 5.6          | 关键机制与参数敏感性分析        | 37        |
| 5.6.1        | 扰动预算与迭代步数敏感性        | 37        |
| 5.6.2        | 扩散编辑强度敏感性           | 37        |
| 5.6.3        | 候选预算与搜索控制机制         | 38        |

|              |                          |           |
|--------------|--------------------------|-----------|
| 5.6.4        | 扩散净化参数敏感性 . . . . .      | 38        |
| 5.6.5        | 轻量输入变换参数敏感性 . . . . .    | 39        |
| 5.6.6        | 生成器基线修正与目标设定边界 . . . . . | 40        |
| 5.7          | 跨模型分析 . . . . .          | 41        |
| 5.8          | 实验总结 . . . . .           | 42        |
| <b>第 6 章</b> | <b>总结与展望</b>             | <b>43</b> |
| 6.1          | 工作总结 . . . . .           | 43        |
| 6.2          | 不足与展望 . . . . .          | 44        |
|              | <b>致谢</b>                | <b>45</b> |

# 摘要

随着生成式人工智能（Artificial Intelligence Generated Content, AIGC）在图像生成与编辑中的快速普及，面向判别模型的对抗威胁已不再局限于  $L_p$  范数约束下的微小像素扰动。攻击者可借助扩散模型等生成式模型对输入进行语义与风格重构，生成视觉自然但可诱导误分类的无限制对抗样本。该攻击更贴近真实 AIGC 内容变换流程，也使现有评估平台在攻击生成、防御净化、质量度量与实验复现方面暴露不足。

针对上述问题，本文围绕扩散式无限制对抗样本的生成、筛选和评估展开研究，提出一种基于 Stable Diffusion img2img 的候选搜索式攻击评估方法 DIFF-SEARCH。该方法将生成模型视为语义编辑算子，在给定提示词、编辑强度和候选预算的条件下生成候选集合；随后利用目标分类器判定攻击成功性，并采用“先成功、再择优”的策略在成功候选中选择感知差异更充分的样本。为降低无效计算，方法进一步引入批内早停和失败回退机制；为避免仅用攻击成功率描述无限制攻击，本文同时定义攻击成功率、干净正确样本攻击成功率、鲁棒准确率、LPIPS、CLIP Score、FID、延迟和资源占用等联合评估指标。工程系统在本文中作为方法实现与实验复现载体，而不是论文叙事的中心。

本文基于 ImageNet 子集实验组织和分析结果。传统攻击使用 500 张图像，扩散攻击使用 200 张图像，主要实验采用 3 个随机种子（42、123、456）报告均值与标准差。结果显示：在 ResNet-50 上，FGSM 总体 ASR 为 40.60%，PGD 为  $92.40\% \pm 0.72\%$ ，扩散攻击为 100.00%；仅统计干净正确样本时，三者 ASR 分别为 42.89%、 $99.68\% \pm 0.14\%$  和 100.00%。候选数量和编辑强度消融表明，扩散候选搜索的影响不只体现在攻击成功率，还体现在查询预算、感知变化与样本自然性之间的权衡。防御实验进一步表明，JPEG、高斯模糊、位深度缩减和扩散净化面对扩散式编辑攻击时鲁棒准确率均不超过 1.50%；扩散净化虽可在部分传统攻击下恢复 20%–25% 的鲁棒准确率，却带来约 138 秒延迟和约 57% 干净准确率下降。这些结果说明，扩散式无限制攻击需以“生成机制–候选选择–多维指标”共同分析，单一平台功能或单一 ASR 数值不足以描述其安全影响。

**关键词：**对抗样本；AIGC 安全；扩散模型；无限制攻击；鲁棒性评估

**中图分类号：**TP391.41

# Abstract

With the rapid deployment of AIGC systems, adversarial threats against vision classifiers are no longer limited to small  $L_p$ -bounded pixel perturbations. Diffusion-based editing can produce unrestricted adversarial examples that remain visually natural yet cause misclassification, exposing gaps in current attack generation, defense purification, quality measurement, and reproducibility practices.

This thesis studies the generation, selection, and evaluation of diffusion-based unrestricted adversarial examples. It formulates a Stable-Diffusion-img2img candidate-search method, DIFFSEARCH, which treats the diffusion model as a semantic editing operator. Given a prompt, edit strength, guidance scale, and candidate budget, the method generates candidate adversarial images, checks their success using the target classifier, and selects the successful candidate with stronger perceptual variation. Batch-level early stopping and fallback generation reduce wasted queries, while a multi-dimensional protocol reports ASR, clean-correct ASR, robust accuracy, LPIPS, CLIP Score, FID, latency, and resource cost. The implemented system serves as the reproducible carrier of the method rather than the central contribution.

The thesis organizes and analyzes results on an ImageNet subset. Traditional attacks use 500 images, diffusion attacks use 200 images, and major runs report mean and standard deviation over three seeds (42, 123, 456). On ResNet-50, FGSM reaches 40.60% ASR, PGD  $92.40\% \pm 0.72\%$ , and the diffusion attack 100.00%; on clean-correct samples the values are 42.89%,  $99.68\% \pm 0.14\%$ , and 100.00%. Candidate-number and strength ablations show that diffusion search affects not only success rate but also query budget, perceptual change, and naturalness. Robust accuracy stays no higher than 1.50% under JPEG, Gaussian blur, bit-depth reduction, or diffusion purification. These results suggest analyzing diffusion-based unrestricted attacks through generation mechanism, candidate selection, and multi-dimensional evidence rather than a single ASR number.

**Keywords:** Adversarial Examples; AIGC Security; Diffusion Models; Unrestricted Attack; Robustness Evaluation

**CLC code:** TP391.41

# 第 1 章 绪论

## 1.1 研究背景与意义

近年来，生成式人工智能（Artificial Intelligence Generated Content, AIGC）技术在图像生成、编辑和内容生产等场景中快速发展。以扩散模型（Diffusion Models）<sup>[2]</sup>和生成对抗网络（Generative Adversarial Networks, GAN）为代表的深度生成模型，已在图像合成<sup>[2]</sup>、文本生成、视频创作等任务中得到广泛应用。Stable Diffusion<sup>[2]</sup>等开源模型能够根据文本提示生成高分辨率图像，推动了生成式模型在数字内容创作、智能设计和虚拟现实等场景中的使用。

与此同时，AIGC 技术的应用也带来了新的安全问题。在对抗机器学习（Adversarial Machine Learning）领域，自 Szegedy 等人<sup>[2]</sup>于 2014 年发现深度神经网络容易受到对抗样本攻击以来，对抗鲁棒性研究已经成为深度学习安全领域的重要议题。传统的对抗样本研究通常假设攻击者在  $L_p$  范数约束下对输入添加人眼不可感知的微小扰动，例如快速梯度符号法（FGSM）<sup>[2]</sup>、投影梯度下降攻击（PGD）<sup>[2]</sup>、C&W 攻击<sup>[2]</sup>等方法。这类攻击设定便于数学建模和标准化基准测试，但难以覆盖 AIGC 场景中基于生成式编辑的攻击形态。

在 AIGC 场景中，攻击者可以利用生成模型在语义、结构、风格、纹理、提示词等多个维度重新构造输入图像，使得对抗样本不再局限于“像素级微小扰动”，而是表现为视觉上自然、接近真实内容变换的“无限制对抗样本”（Unrestricted Adversarial Examples）<sup>[2, 3]</sup>。例如，AdvDiffuser<sup>[2]</sup>利用 Stable Diffusion 模型的图像编辑能力生成视觉上自然但能欺骗分类器的对抗图像，这类攻击不需要满足固定的  $L_p$  范数约束。无限制对抗样本削弱了基于像素扰动假设的防御和评估方法的适用性，也对现有对抗鲁棒性评估体系提出了新的要求。

从应用安全角度来看，无限制对抗样本可能影响多个安全敏感领域。在自动驾驶场景中，攻击者可以利用生成模型构造看似正常但能误导感知系统的交通标志或道路场景，导致车辆做出错误决策。在医学影像诊断领域，生成式对抗样本可能干扰计算机辅助诊断系统的判断结果，造成误诊或漏诊。在安防监控和人脸识别系统中，基于生成模型的对抗攻击可能绕过身份验证和活体检测机制。在内容审核和版权保护领域，AIGC 生成的对抗样本也可能影响内容检测系统的可靠性。因此，本文将研究重点放在扩散式无限制对抗样本的生成机制、候选选择策略和鲁棒性评估方法上，通过可复现实验系统承载实验，而不是把系统工



程本身作为唯一研究对象。

## 1.2 研究现状与问题

### 1.2.1 对抗攻击研究现状

对抗攻击方法经历了从像素级扰动到生成式变换的发展过程。在传统梯度攻击方面, Goodfellow 等人<sup>[2]</sup>于 2015 年提出了快速梯度符号法 (FGSM), 该方法通过计算损失函数相对于输入的梯度方向符号, 沿该方向添加固定幅度的扰动来生成对抗样本, 是较早的高效白盒攻击方法之一。Madry 等人<sup>[2]</sup>于 2018 年提出了投影梯度下降 (PGD) 攻击, 将 FGSM 扩展为多步迭代形式, 通过反复计算梯度并投影到扰动范围内增强攻击效果。Carlini 和 Wagner<sup>[2]</sup>提出了 C&W 攻击, 通过优化目标搜索能够改变模型预测结果的较小扰动。这些传统方法均在  $L_p$  范数约束下工作, 生成的对抗扰动通常人眼不可感知但具有明确的数学边界。

随着深度生成模型的发展, 基于生成式方法的对抗攻击逐渐出现。Song 等人<sup>[2]</sup>较早提出利用条件生成模型构造不受小范数扰动约束的对抗样本。Bai 等人<sup>[2]</sup>提出了 AI-GAN, 利用 GAN 架构训练生成器网络, 以原始样本为输入直接生成对抗样本。Chen 等人<sup>[2]</sup>提出了 AdvDiffuser, 利用 Stable Diffusion<sup>[2]</sup>的图像编辑能力, 通过在扩散过程中注入目标分类器的梯度信号来引导生成对抗样本, 实现不受固定  $L_p$  范数约束的攻击。Xie 等人<sup>[2]</sup>进一步研究了面向 AIGC 检测系统的对抗攻击, 提出“Take Fake as Real”方法。这类生成式攻击能够产生视觉上更自然的对抗样本, 对传统基于扰动检测的防御方法提出了新的挑战。

### 1.2.2 对抗防御研究现状

对抗防御方法主要包括对抗训练、输入净化和认证防御三大类技术路线。

对抗训练<sup>[2, 3]</sup>是目前最为广泛使用的鲁棒性增强方法。其核心思想是在模型训练过程中, 对每个训练样本生成对抗样本并将其加入训练集, 从而使模型学习到对扰动具有鲁棒性的特征表示。然而, 对抗训练存在计算开销大 (通常需要 10 倍以上的训练时间)、对新型攻击的泛化能力有限、以及可能导致模型在干净样本上的准确率下降等问题。

输入净化方法通过在模型推理之前对输入进行预处理来减弱潜在的对抗扰动。Song 等人<sup>[2]</sup>提出了 PixelDefend, 利用 PixelCNN 生成模型将对抗样本投影回干净数据的自然流形。Nie 等人<sup>[2]</sup>提出了 DiffPure 方法, 利用扩散模型的去噪过程来净化对抗样本, 其核心思想是对抗扰动可以被前向扩散过程中添加的噪声掩盖, 然后通过反向去噪过程恢复近似干净的图像。JPEG 有损压缩、高斯模糊滤波、位深度缩减等轻量级防御方法通过破坏图像中的高频信息来降低对抗

扰动的效果，具有计算效率高、部署简便等优势，但面对语义级攻击时防御能力有限。

认证防御方法以 Cohen 等人<sup>[2]</sup>提出的随机平滑方法为代表，能够为模型在给定扰动半径内的预测一致性提供可证明保证。然而，认证防御通常会降低干净样本准确率，且主要适用于  $L_2$  范数约束下的扰动场景，难以直接应用于不存在明确扰动范数约束的无限制对抗样本防御。此外，认证防御的推理过程需要对每个输入进行多次随机采样和投票，推理效率低于常规分类模型，限制了其在实时系统中的部署。

### 1.2.3 鲁棒性评估工具研究现状

在对抗鲁棒性评估平台方面，学术界和工业界已经开发了若干具有代表性的工具。RobustBench<sup>[2]</sup>提供了标准化的评估协议、预训练鲁棒模型和公开排行榜，但主要面向  $L_p$  范数攻击场景，缺少对生成式攻击和 AIGC 场景的支持。CleverHans<sup>[2]</sup>是一个模块化的对抗机器学习 Python 库，提供了多种攻击和防御算法实现，但缺乏端到端的实验流程编排和自动化报告生成功能。Adversarial Robustness Toolbox (ART)<sup>[2]</sup>是 IBM 开发的对抗机器学习工具箱，覆盖攻击、防御、检测和认证等多个方面，但其架构偏重算法 API 层面，缺乏面向 AIGC 场景的专用评估协议和自动化 workflow。

综合分析，现有评估平台主要存在以下四方面局限性：（1）攻击类型局限于  $L_p$  范数约束下的像素级扰动攻击，缺乏对 AIGC 生成式语义级攻击的系统性支持；（2）评估维度单一，主要关注鲁棒准确率和攻击成功率，缺少感知质量 (LPIPS)、语义一致性 (CLIP Score) 和分布距离 (FID) 等多维评估指标；（3）实验流程自动化程度不足，缺乏从配置到报告的端到端编排能力；（4）对扩散净化等新型防御方法的集成支持有限。

## 1.3 研究目标与内容

针对上述问题，本文的研究目标是围绕 AIGC 场景下扩散式无限制对抗样本的生成与评估，形成一套可解释、可复现、可扩展的候选搜索方法。本文并不声称复现 AdvDiffuser 的梯度注入或对抗修复等全部机制，而是在现有 Stable Diffusion img2img 管线基础上，将候选生成、成功判定、预算控制、失败回退和多指标评估组织为一个清晰的算法流程。平台实现作为该方法的工程载体，用于保证实验配置、随机种子、指标和样本产物能够被完整回溯。具体研究内容包括以下五个方面：

- （1）扩散式候选生成建模：将 Stable Diffusion img2img 管线形式化为从原始样本、提示词、编辑强度、引导系数和随机噪声到候选图像的条件生成算子，

分析其与传统  $L_p$  约束攻击在搜索空间、可解释性和质量约束上的差异。

- (2) **候选搜索与筛选策略设计**: 提出基于候选预算的扩散攻击流程, 包含非目标/目标两类成功判定、批内早停、失败回退和 LPIPS 择优规则, 使扩散攻击不只是单次图像编辑, 而是具有明确预算和选择准则的搜索算法。
- (3) **多维评估协议构建**: 构建覆盖攻击效果、感知变化、语义一致性、分布差异、防御恢复和资源代价的评估体系, 避免仅用 ASR 或单一鲁棒准确率解释无限制攻击。
- (4) **前沿算法路线梳理与可扩展实现接口**: 结合 AdvDiff、AdvDiffuser、SCA、AnyAttack 和扩散净化等近期工作, 讨论梯度引导、扩散反演、语义一致性约束和视觉语言模型攻击等后续可补实现路线, 并在工程结构中保留可扩展接口。
- (5) **实验验证与分析**: 基于 ImageNet<sup>[2]</sup> 子集整理传统攻击、扩散攻击、候选数量、编辑强度、攻防交叉和跨模型实验结果, 重点分析扩散式候选搜索的攻击特征和现有防御在生成式编辑攻击下的局限。

## 1.4 论文组织结构

本文共分为六章, 各章内容安排如下:

第 1 章为绪论, 介绍本文的研究背景与意义、研究现状与问题、研究目标与内容, 以及论文的组织结构。

第 2 章为相关工作, 系统回顾传统对抗攻击与防御、无限制对抗样本与生成式攻击、AIGC 安全检测与防御、扩散式攻击前沿路线和鲁棒性评估工具的发展, 分析现有工作的优势与不足, 明确本文的研究切入点。

第 3 章为扩散式无限制对抗样本生成与评估方法, 形式化定义攻击问题、候选生成算子、成功判定、早停回退、候选选择和多维评估指标, 并给出算法流程和图解。

第 4 章为算法实现与实验支撑系统, 介绍分类器、生成模型、攻击与防御模块、评估指标和配置化运行机制如何支撑第 3 章提出的方法。

第 5 章为实验与分析, 展示传统扰动基线、DIFFSEARCH 候选搜索、同类攻击方法比较、攻防交互矩阵、关键机制敏感性和跨模型结构分析结果, 并进行系统的定性和定量讨论。

第 6 章为总结与展望, 总结本文的主要方法、实验结论和实现边界, 讨论当前工作的局限性并展望未来的研究方向。

## 第 2 章 相关工作

本章围绕“扩散式无限制对抗样本生成与评估”这一核心技术展开。传统  $L_p$  攻击与输入净化方法主要作为对照基线介绍；无限制对抗样本、生成式攻击和扩散净化构成本文方法的直接背景；鲁棒性评估工具则用于说明本文为何需要把候选预算、样本质量、防御副作用和资源代价纳入同一实验协议。相比宽泛罗列 AIGC 安全问题，本章重点放在与本文候选搜索方法直接相关的生成机制、约束假设和评价口径上。

### 2.1 传统对抗攻击与防御

传统对抗攻击以  $L_p$  范数约束为核心假设：攻击者在原始图像附近寻找一个小扰动  $\delta$ ，使  $x + \delta$  仍位于允许扰动集合内但能改变模型预测。Szegedy 等人<sup>[2]</sup>最早揭示了深度网络对微小扰动的脆弱性；Goodfellow 等人<sup>[2]</sup>提出的 FGSM 用单步梯度符号快速生成扰动；Madry 等人<sup>[2]</sup>提出的 PGD 通过多步迭代和投影获得更强的一阶白盒攻击；Carlini 和 Wagner<sup>[2]</sup>进一步将攻击表述为带约束优化问题。这些方法具有清晰数学边界和成熟评测传统，因此本文将 FGSM 和 PGD 作为传统扰动基线，用于对比扩散式候选搜索在攻击空间、视觉变化和防御响应上的差异。

防御侧同样形成了若干常用路线。对抗训练<sup>[2]</sup>通过在训练阶段纳入对抗样本提升模型鲁棒性，但通常计算成本高，并可能牺牲干净样本准确率。输入净化方法在推理前对输入进行变换，例如 PixelDefend<sup>[2]</sup>使用生成模型投影输入，JPEG 压缩、高斯模糊和位深度缩减则通过破坏高频扰动或量化像素值降低传统扰动效果。认证防御以随机平滑<sup>[2]</sup>为代表，能在特定范数半径内给出可证明保证，但较难直接覆盖没有固定扰动半径的无限制攻击。因此，本文实验只选取轻量输入变换和扩散净化作为可直接部署的防御对照，并把防御副作用作为重要指标。

### 2.2 无限制对抗样本与生成式攻击

传统对抗样本受  $L_p$  范数约束，其扰动幅度有明确的数学边界，通常被设计为人眼不可感知的微小扰动。与此不同，无限制对抗样本的早期研究已经将攻击者从小范数扰动约束中释放出来，允许构造任意清晰且语义明确的输入<sup>[2]</sup>，或

利用生成模型在数据流形上合成对抗样本<sup>[2]</sup>。在 AIGC 场景下，这一思想进一步表现为通过语义、风格、纹理或局部结构层面的自然变换生成能够欺骗目标模型的样本。本文标题中的“无限制”采用这一工程定义：评价重点从像素扰动半径转向攻击是否保持视觉自然性、语义可解释性以及是否能在 AIGC 生成/编辑流程中复现。

表 2-1 进一步对比了传统约束攻击与几类生成式无限制攻击的差异。可以看到，现有工作正从 GAN 生成器、扩散采样引导，逐步走向扩散反演、语义一致性约束和视觉语言模型攻击。本文已实现的 DIFFSEARCH 属于“img2img 候选搜索”路线，它没有使用梯度注入或大规模生成器预训练，但提供了明确的候选预算、成功判定、早停回退和多指标评估机制，可作为后续接入更强攻击算法的基础流程。

### 2.2.1 基于 GAN 的生成式方法

Bai 等人<sup>[2]</sup>提出的 AI-GAN 是基于 GAN 框架的生成式对抗攻击方法的典型代表。其核心思想是训练一个专用的生成器网络  $G$ ，以原始干净样本  $x$  为输入，直接输出对应的对抗样本  $G(x)$ 。生成器通过联合优化对抗损失（确保攻击成功）和感知损失（保持视觉自然性）来进行训练。AI-GAN 的优势在于一旦生成器训练完成，生成对抗样本的推理速度极快，适合大规模攻击场景。但其局限性在于生成器的攻击能力受限于训练数据的分布，对未见过的图像类别可能泛化不足。

### 2.2.2 基于扩散模型的生成式方法

扩散模型<sup>[2]</sup>是近年来常用的深度生成模型之一，其核心思想是定义一个前向扩散过程逐步向数据添加噪声，再学习一个反向去噪过程从噪声恢复数据。

**前向扩散过程。**给定数据样本  $x_0 \sim q(x_0)$ ，前向过程在  $T$  步中逐步添加高斯噪声：

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I}), \quad t = 1, \dots, T, \quad (2.1)$$

其中  $\{\beta_t\}_{t=1}^T$  为预定义的噪声调度。令  $\alpha_t = 1 - \beta_t$ ， $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ ，则任意时刻  $t$  的加噪样本满足闭式分布：

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad (2.2)$$

亦可写为重参数化采样形式：

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}). \quad (2.3)$$

表 2-1 无限制对抗样本相关工作的对比

| 工作类型                       | 约束形式       | 生成机制                | 主要评价重点                           | 与本文关系        |
|----------------------------|------------|---------------------|----------------------------------|--------------|
| FGSM/PGD                   | $L_p$ 范数球  | 梯度扰动                | ASR、RA                           | 作为传统基线       |
| 无限制挑战设定 <sup>[2]</sup>     | 无固定范数球     | 任意清晰输入              | 覆盖率与高置信误分类                       | 概念与评测设定参考    |
| 生成模型无限制攻击 <sup>[2]</sup>   | 无固定范数球     | 条件生成模型潜空间搜索         | 攻击成功与样本似然                        | 早期生成式路线      |
| AI-GAN <sup>[2]</sup>      | 无固定范数球     | GAN 生成器             | 攻击成功与视觉自然性                       | 生成式攻击参考      |
| AdvDiff <sup>[2]</sup>     | 无固定范数球     | 扩散反向过程对抗引导          | 攻击性能与生成质量                        | 后续梯度引导路线     |
| AdvDiffuser <sup>[2]</sup> | 无固定范数球     | Stable Diffusion 编辑 | 自然图像编辑攻击                         | 扩散攻击方法启发     |
| SCA <sup>[2]</sup>         | 语义一致性约束    | DDPM 反演+MLLM 语义引导   | 语义保持与采样效率                        | 后续语义约束路线     |
| AnyAttack <sup>[2]</sup>   | 目标图像/多模态任务 | 自监督目标攻击生成器          | VLM 迁移攻击                         | 后续多模态扩展路线    |
| AIGC 检测绕过 <sup>[2]</sup>   | 任务相关自然性约束  | 真实感生成/变换            | 检测器绕过率                           | 说明 AIGC 安全场景 |
| 本文方法                       | 配置化候选预算    | SD img2img 候选搜索     | ASR、ASR-C、RA、LPIPS、CLIP、FID、资源开销 | 已实现验证主线      |

**反向去噪过程。**DDPM 用参数化网络  $\epsilon_\theta(x_t, t)$  预测噪声分量，反向转移分布为：

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \tilde{\beta}_t \mathbf{I}), \quad (2.4)$$

其中均值由噪声预测网络给出：

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right), \quad (2.5)$$

方差为  $\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$ 。训练目标为简化的噪声预测损失：

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]. \quad (2.6)$$

**潜空间扩散 (Latent Diffusion)。**Stable Diffusion<sup>[2]</sup> 将上述过程迁移到自编码器的潜空间中执行。设编码器为  $\mathcal{E}$ ，解码器为  $\mathcal{D}$ ，潜变量  $z_0 = \mathcal{E}(x_0)$ ，则扩散过程在  $z$  空间进行，生成结果为  $\hat{x}_0 = \mathcal{D}(\hat{z}_0)$ 。img2img 管线对输入图像先编码再加噪到时刻  $t_s = \lfloor s \cdot T \rfloor$  ( $s$  为编辑强度)，然后从  $z_{t_s}$  开始执行条件去噪，最终解码得到编辑后的图像。

Chen 等人<sup>[2]</sup> 提出的 AdvDiffuser 是基于扩散模型的生成式对抗攻击方法的重要代表。AdvDiffuser 利用 Stable Diffusion<sup>[2]</sup> 提供的 img2img 管线，在扩散过程中注入目标分类器的梯度信号，引导生成过程产生既能有效欺骗分类器又保持视觉自然性的对抗样本。AdvDiffuser 的生成过程可以概括表示为：

$$x_{\text{adv}} = \text{SD}_{\text{img2img}}(x, p, s; \nabla_x \mathcal{L}_{\text{target}}) \quad (2.7)$$

其中  $p$  为引导文本提示 (text prompt)， $s$  为变换强度参数 (strength)， $\nabla_x \mathcal{L}_{\text{target}}$  为目标分类器提供的梯度信号。变换强度  $s$  控制了生成图像与原始图像之间的偏离程度： $s$  值越大，生成结果与原图差异越大，攻击成功的概率也越高，但同时图像的语义内容可能发生较大变化。

Xie 等人<sup>[2]</sup> 进一步研究了面向 AIGC 检测系统的对抗攻击，提出了“Take Fake as Real”方法，通过生成具有真实图像统计特征的对抗本来规避 AI 生成内容检测系统的检测。这项工作表明 AIGC 检测系统自身也存在被对抗攻击绕过的风险，进一步说明了构建系统化攻防验证体系的必要性。

### 2.2.3 扩散反演与多模态前沿路线

除直接调用 img2img 管线外，近期工作还在三个方向上推进无限制攻击算法。第一类是对扩散采样过程加入更显式的对抗引导，例如 AdvDiff<sup>[2]</sup> 将目标分类器梯度整合到反向生成过程中，使攻击目标不仅依赖随机候选搜索，而是直

接影响去噪轨迹。第二类是使用扩散反演获得与原图对应的潜变量或噪声轨迹，再在潜空间中执行语义保持的编辑；SCA<sup>[?] ]</sup>即利用 DDPM 反演和多模态大模型提供语义引导，以减少无限制攻击中常见的类别语义漂移。第三类是从传统分类模型扩展到视觉语言模型，AnyAttack<sup>[?] ]</sup>将任意图像作为目标，构造可迁移到图文检索、图像描述和多模态分类等任务的目标攻击。

这些前沿算法与本文已实现方法存在清晰的递进关系。本文当前实验采用候选搜索而非梯度引导，优势是实现成本低、可复现性强、与现有 Stable Diffusion img2img 接口兼容；不足是目标攻击能力弱、语义约束较粗、采样效率受候选预算限制。第3章因此把本文方法抽象为“生成算子 + 候选集合 + 选择准则”的形式，使未来接入 AdvDiff 式梯度引导、SCA 式反演轨迹或 AnyAttack 式多模态目标时，只需替换候选生成算子和成功判定函数，而无需改变整体评估协议。

## 2.3 AIGC 安全检测与防御

### 2.3.1 扩散净化方法

Nie 等人<sup>[?] ]</sup>提出的 DiffPure 方法是利用扩散模型进行对抗防御的开创性工作。DiffPure 的核心洞察是：对抗扰动作为人为添加的非自然信号，可以被扩散模型的前向过程中添加的高斯噪声所淹没，随后通过反向去噪过程恢复出近似干净的图像。DiffPure 的处理过程可以分为两个步骤：

$$x_t = \sqrt{\bar{\alpha}_t} x_{\text{adv}} + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \hat{x} = \text{REVERSEDDPM}(x_t; t \rightarrow 0), \quad (2.8)$$

其中  $\bar{\alpha}_t$  为扩散噪声调度中的累计信号保留系数， $\epsilon \sim \mathcal{N}(0, I)$  为标准高斯噪声，REVERSEDDPM 表示从时刻  $t$  到 0 的反向去噪过程。噪声水平的选择至关重要：过小的噪声无法有效掩盖对抗扰动，过大的噪声则会破坏图像的原始语义信息。DiffPure 在面对传统  $L_p$  范数攻击时展现了良好的防御效果，但其高计算开销（需要多步去噪迭代）和对语义级攻击的防御局限性是主要问题。

Kang 等人<sup>[?] ]</sup>提出了 Diffender 方法，专门针对对抗补丁攻击（Adversarial Patch Attack）设计了基于扩散模型的防御方案，展现了扩散模型在对抗防御领域的广泛应用潜力。

### 2.3.2 轻量级防御方法

轻量级防御方法通过简单而高效的输入变换来削弱对抗扰动的效果，具有计算开销低、部署简便的优势：

- **JPEG 压缩**：JPEG 有损压缩通过离散余弦变换（DCT）和量化操作来压缩图像数据，在此过程中高频信息（包括对抗扰动中的高频分量）会被部分



丢弃。压缩质量因子（quality factor）越低，信息丢失越多，对抗动的破坏效果越强，但同时也会降低图像的视觉质量。

- **高斯模糊**：高斯模糊通过对图像应用高斯核进行卷积滤波，实现低通滤波的效果，平滑图像中的高频成分。核大小和标准差共同决定了模糊的程度和范围。
- **位深度缩减**：位深度缩减通过将图像的颜色精度从标准的 8 位（256 级灰度）降低到更少的位数（如 4 位，16 级灰度），对像素值进行粗粒度量化。这种量化操作能够有效破坏像素级别的细微对抗扰动，但对语义级别的大范围变换基本无效。

## 2.4 鲁棒性评估工具与实验协议

对抗鲁棒性研究离不开稳定的评估工具。**RobustBench**<sup>[?]1</sup>提供标准化排行榜和鲁棒模型，适合  $L_p$  范数攻击下的基准比较；**CleverHans**<sup>[?]2</sup>和 **ART**<sup>[?]3</sup>提供丰富的攻击、防御和检测 API，适合作为算法参考实现。本文不把工具覆盖范围作为主贡献，而是从这些生态中抽取评估经验，补充面向扩散式无限制攻击的实验口径：除模型是否被误分类外，还同步记录候选预算、视觉变化幅度、语义一致性、防御副作用和资源代价。这样的协议为后续接入 **AdvDiff**、**AdvDiffuser**、**SCA** 或 **AnyAttack** 等更强候选生成算子提供统一比较口径，也避免把“能否运行某个攻击库”误当作安全结论。

# 第 3 章 扩散式无限制对抗样本生成与评估方法

## 3.1 问题定义

设图像分类器为  $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ ，输入图像为  $x \in [0, 1]^{C \times H \times W}$ ，真实标签为  $y$ 。传统对抗攻击通常在扰动集合  $\mathcal{S}_p(\epsilon) = \{\delta : \|\delta\|_p \leq \epsilon\}$  内搜索  $x + \delta$ ，其优化目标可写为

$$\max_{\delta \in \mathcal{S}_p(\epsilon)} \mathcal{L}(f_\theta(x + \delta), y). \quad (3.1)$$

这种形式适合描述像素级扰动，但难以描述 AIGC 图像编辑中常见的颜色、纹理、局部结构和风格变化。本文将扩散式无限制攻击定义为在生成模型诱导的自然图像邻域中搜索候选样本。

### 3.1.1 候选生成算子的形式化定义

基于第 2 章中 Stable Diffusion 的潜空间扩散框架，本文将 img2img 候选生成过程形式化为四步组合算子。给定原始图像  $x_i$ 、提示词  $p$ 、编辑强度  $s \in (0, 1]$ 、引导系数  $g$  和随机噪声  $\xi_k$ ：

**步骤一：潜空间编码。**使用预训练自编码器将输入映射到潜空间：

$$z_0 = \mathcal{E}(x_i). \quad (3.2)$$

**步骤二：前向加噪。**由编辑强度  $s$  确定加噪起点  $t_s = \lfloor s \cdot T \rfloor$ ，直接从  $z_0$  采样到时刻  $t_s$ ：

$$z_{t_s} = \sqrt{\bar{\alpha}_{t_s}} z_0 + \sqrt{1 - \bar{\alpha}_{t_s}} \xi_k, \quad \xi_k \sim \mathcal{N}(0, \mathbf{I}). \quad (3.3)$$

**步骤三：条件去噪。**在文本提示  $p$  和引导系数  $g$  的控制下，执行  $t_s$  步反向去噪：

$$\hat{z}_0 = \text{DENOISE}_\theta(z_{t_s}, p, g; t_s \rightarrow 0), \quad (3.4)$$

其中去噪过程使用无分类器引导（Classifier-Free Guidance）调节噪声预测。按 diffusers 中 guidance\_scale 的实现口径，若  $g$  表示引导系数，则

$$\hat{\epsilon}_\theta = \epsilon_\theta(z_t, \emptyset, t) + g(\epsilon_\theta(z_t, p, t) - \epsilon_\theta(z_t, \emptyset, t)) = g \epsilon_\theta(z_t, p, t) + (1 - g) \epsilon_\theta(z_t, \emptyset, t). \quad (3.5)$$

若另以  $w$  表示相对条件预测的额外外推强度, 则同一关系也可写为  $(1+w)\epsilon_{\text{cond}} - w\epsilon_{\text{uncond}}$ , 二者满足  $g = 1 + w$ 。

**步骤四：潜空间解码。**将去噪后的潜变量映射回图像空间：

$$x_i^{(k)} = \mathcal{D}(\hat{z}_0). \quad (3.6)$$

将上述四步简记为候选生成算子：

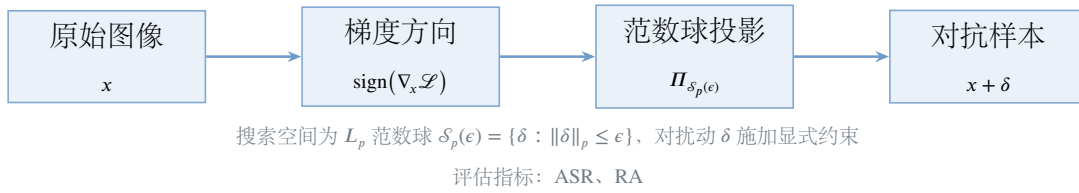
$$x_i^{(k)} = G_\phi(x_i, p, s, g, \xi_k) \triangleq \mathcal{D}\left(\text{DENOISE}_\theta\left(\sqrt{\bar{\alpha}_{t_s}} \mathcal{E}(x_i) + \sqrt{1 - \bar{\alpha}_{t_s}} \xi_k, p, g\right)\right), \quad (3.7)$$

$$k = 1, \dots, N.$$

其中  $N$  是候选预算。与式 (3.1) 不同, 式 (3.7) 没有显式的范数球约束; 其约束由生成模型、提示词、编辑强度和候选预算共同决定。编辑强度  $s$  在此框架中具有明确的物理意义: 它控制加噪深度  $t_s$ ,  $s$  越大则  $z_{t_s}$  中原始图像信息被噪声覆盖越多, 去噪过程拥有更大的自由度重构图像内容, 从而产生更大的语义偏离。因此, 扩散式无限制攻击的核心不只是“生成一张图片”, 而是如何在候选集合中判断攻击成功、控制查询成本、选择最终对抗样本, 并用合适指标解释样本质量和防御效果。

图 3-1 将两类攻击抽象为统一的“输入-处理-候选/扰动-输出”四阶段流程, 并标注各自的搜索空间、约束形式和评估维度, 凸显二者在生成机制和评价口径上的差异。

#### 传统 $L_p$ 约束攻击



#### 扩散式无限制攻击

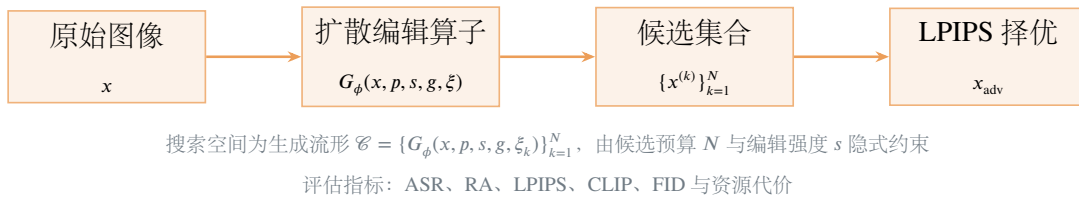


图 3-1 传统  $L_p$  约束攻击与扩散式无限制攻击的算法流程对比

## 3.2 候选搜索算法

本文提出的 **DIFFSEARCH** 方法由候选生成、成功判定、早停控制、失败回退和最终选择五个部分构成。整体流程如图 3-2 所示。图中的扩散编辑器可以是当前实现中的 **Stable Diffusion img2img**，也可以在后续替换为 **AdvDiff** 式梯度引导采样、**SCA** 式扩散反演轨迹或视觉语言模型攻击生成器；只要候选集合和成功判定接口保持一致，评估协议即可复用。

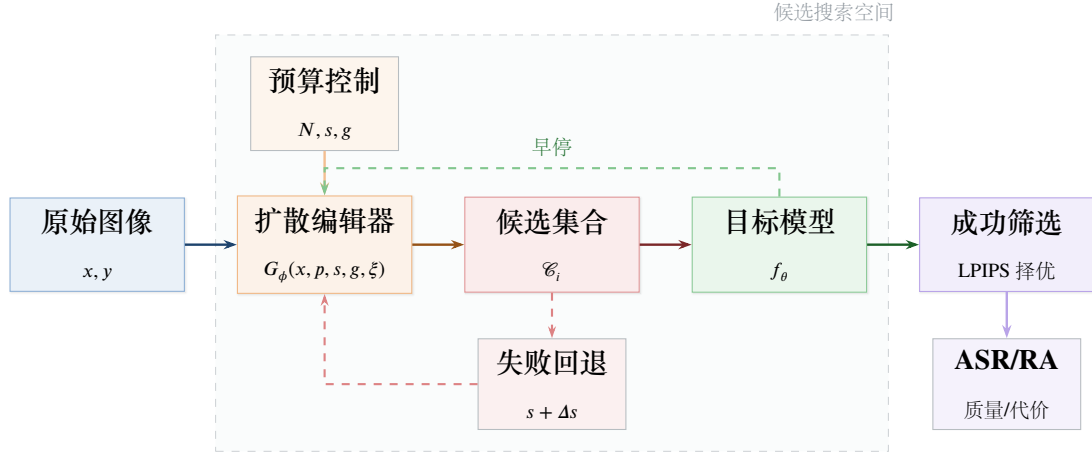


图 3-2 扩散式候选搜索攻击方法示意图

### 3.2.1 成功判定

对于非目标攻击，本文使用“预测发生变化”作为候选成功条件。令原始预测为  $\hat{y}_i = f_\theta(x_i)$ ，候选预测为  $\hat{y}_i^{(k)} = f_\theta(x_i^{(k)})$ ，则成功指示函数为

$$S_i^{(k)} = \text{Ind}[\hat{y}_i^{(k)} \neq \hat{y}_i]. \quad (3.8)$$

对于目标攻击，给定目标类别  $t$ ，成功指示函数改为

$$S_i^{(k)} = \text{Ind}[\hat{y}_i^{(k)} = t]. \quad (3.9)$$

本文默认评估采用非目标设定。目标类别设定作为边界分析结果保留，其较低成功率说明：直接复用通用提示词和 **img2img** 候选搜索并不足以稳定命中特定类别，后续若要强化目标类别命中，应引入类别条件提示、梯度引导或视觉语言模型目标对齐机制。

### 3.2.2 候选选择准则

对每个样本  $x_i$ ，算法维护成功候选集合

$$\mathcal{C}_i^+ = \left\{ x_i^{(k)} \mid S_i^{(k)} = 1, k \leq N_i \right\}, \quad (3.10)$$

其中  $N_i$  为该样本实际消耗的候选次数。若  $\mathcal{C}_i^+$  非空，本文选择 LPIPS 距离最大的成功候选作为最终对抗样本：

$$x_{i,\text{adv}} = \arg \max_{z \in \mathcal{C}_i^+} d_{\text{LPIPS}}(x_i, z). \quad (3.11)$$

该准则的目的不是追求最小扰动，而是体现无限制攻击与传统  $L_p$  攻击的差异：在保持生成图像自然性的前提下，成功候选中更大的感知变化往往意味着更充分的语义或纹理编辑，也更能暴露输入净化类防御的局限。为了避免误解，本文不把 LPIPS 最大化解释为通用最优目标；它是当前实验中用于选择代表性成功候选的工程准则。

### 3.2.3 早停与失败回退

扩散模型推理成本较高，如果固定为所有样本生成  $N$  个候选，会浪费大量已经成功样本上的查询。因此，DIFFSEARCH 在批次层面采用早停机制：每轮候选生成后更新所有样本的成功标记，若批次中每个样本都至少出现一个成功候选，则停止后续候选生成。早停机制使配置中的  $N$  成为候选上限而不是每个样本的固定查询次数。

若部分样本在主循环结束后仍未成功，算法对失败子集执行回退生成。回退阶段将编辑强度提高到

$$s_{\text{fb}} = \min(1, s + \Delta s), \quad (3.12)$$

并最多额外尝试  $M$  个候选。回退只作用于失败样本，因此不会改变已经成功样本的候选选择。该机制体现了攻击预算的分层思想：主循环使用保守强度保证整体自然性，回退阶段只在必要时提高编辑幅度以增加成功概率。

**Algorithm 1** DIFFSEARCH: 扩散式候选搜索攻击

**Require:** 图像批次  $X$ , 标签  $Y$ , 分类器  $f_\theta$ , 扩散编辑器  $G_\phi$ , 候选预算  $N$ , 回退预算  $M$

**Ensure:** 对抗样本  $X_{\text{adv}}$ , 成功标记  $S$ , 查询次数  $Q$

```

1: 初始化  $X_{\text{adv}} \leftarrow X$ ,  $S_i \leftarrow \text{false}$ ,  $Q_i \leftarrow 0$ , 最佳 LPIPS 分数  $B_i \leftarrow -1$ 
2: 计算原始预测  $\hat{Y} \leftarrow f_\theta(X)$ 
3: for  $k = 1$  to  $N$  do
4:   生成候选  $Z \leftarrow G_\phi(X, p, s, g, \xi_k)$ 
5:   计算候选预测  $\hat{Z} \leftarrow f_\theta(Z)$  和 LPIPS 分数  $D \leftarrow d_{\text{LPIPS}}(X, Z)$ 
6:   for all 样本  $i$  do
7:      $Q_i \leftarrow Q_i + 1$ 
8:     if SUCCESS( $\hat{Z}_i, \hat{Y}_i, Y_i$ ) and  $D_i > B_i$  then
9:        $X_{\text{adv},i} \leftarrow Z_i$ ,  $S_i \leftarrow \text{true}$ ,  $B_i \leftarrow D_i$ 
10:    end if
11:  end for
12:  if 所有  $S_i$  均为 true then
13:    break
14:  end if
15: end for
16: for all 仍失败的样本  $i$  do
17:   for  $m = 1$  to  $M$  do
18:    用  $s_{\text{fb}} = \min(1, s + \Delta s)$  生成回退候选并更新  $X_{\text{adv},i}, S_i, Q_i$ 
19:    if  $S_i$  为 true then
20:      break
21:    end if
22:  end for
23: end for
24: return  $X_{\text{adv}}, S, Q$ 

```

## 3.2.4 查询复杂度分析

算法 1 的查询开销取决于早停和回退机制的交互。设批次中共有  $n$  个样本，单个候选对生成器的查询成本为  $c_G$ （含扩散去噪），对分类器的查询成本为  $c_f$ ，LPIPS 评估成本为  $c_L$ 。定义单个样本在第  $k$  轮首次成功的概率为  $p_k(s)$ ，则：

**最优情况。**若所有样本在第 1 轮即成功（ $p_1(s) = 1$ ），早停立即触发：

$$Q_{\text{best}} = n \cdot (c_G + c_f + c_L). \quad (3.13)$$

**最坏情况。**若所有样本在主循环均失败，且回退阶段全部耗尽预算：

$$Q_{\text{worst}} = n \cdot (N + M) \cdot (c_G + c_f + c_L). \quad (3.14)$$

**期望情况。**设样本独立，令每轮成功概率为常数  $p$ 。若单个样本的首次成功轮次为几何随机变量  $T_i$ ，主循环最多查询  $N$  轮，则截断查询轮次  $K_i = \min(T_i, N)$  的期望为

$$\mathbb{E}[K_i] = \sum_{k=1}^N \Pr(T_i \geq k) = \frac{1 - (1-p)^N}{p}. \quad (3.15)$$

在批内早停机制下，主循环的实际总轮次等于  $K_{\max} = \max_i K_i$ 。对  $n$  个独立样本，批次期望轮次为

$$\mathbb{E}[K_{\max}] = \sum_{k=1}^N [1 - (1 - (1-p)^{k-1})^n]. \quad (3.16)$$

该式反映了批量早停的关键特征：当每轮成功概率  $p$  较高时（例如本文实验中  $s = 0.7$  对应约  $p \geq 0.9$ ），即使批量较大，期望轮次也通常低于候选上限  $N$ 。回退阶段只对主循环后仍失败的样本执行。若回退阶段单轮成功概率为  $p_{\text{fb}}$ ，则期望额外查询可近似写为  $(1-p)^N \cdot n \cdot \{1 - (1-p_{\text{fb}})^M\}/p_{\text{fb}}$ ；在主循环成功率较高时，这一项相对较小。

### 3.3 多维评估协议

无限制攻击的评价不能只看误分类比例。一方面，攻击样本可能通过过强编辑改变原始语义，从而获得形式上的高 ASR；另一方面，防御方法可能通过严重破坏干净输入来提高部分鲁棒准确率。因此，本文使用攻击效果、样本质量、防御效果和资源代价四组指标共同刻画方法表现。

攻击成功率定义为

$$\text{ASR} = \frac{1}{n} \sum_{i=1}^n S_i. \quad (3.17)$$

由于部分干净样本在攻击前已经被模型误分类，本文同时报告仅干净正确样本上的攻击成功率：

$$\text{ASR-C} = \frac{\sum_{i=1}^n S_i \cdot \text{Ind}[f_{\theta}(x_i) = y_i]}{\sum_{i=1}^n \text{Ind}[f_{\theta}(x_i) = y_i]}. \quad (3.18)$$

对于防御函数  $D_{\psi}$ ，鲁棒准确率和干净准确率下降分别定义为

$$\text{RA}(D_\psi) = \frac{1}{n} \sum_{i=1}^n \text{Ind}[f_\theta(D_\psi(x_{i,\text{adv}})) = y_i], \quad (3.19)$$

$$\Delta_{\text{clean}}(D_\psi) = \text{Acc}(X) - \text{Acc}(D_\psi(X)). \quad (3.20)$$

样本质量维度包含 LPIPS<sup>[21]</sup>、CLIP Score<sup>[21]</sup> 和 FID<sup>[21]</sup> 三组指标，每组具有明确的数学定义。

**LPIPS (Learned Perceptual Image Patch Similarity)**<sup>[21]</sup>。设预训练特征网络  $\Phi$  在第  $l$  层输出归一化特征图  $\hat{\Phi}^l(x) \in \mathbb{R}^{H_l \times W_l \times C_l}$ ，则 LPIPS 距离定义为

$$d_{\text{LPIPS}}(x, x') = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \|w_l \odot (\hat{\Phi}_{h,w}^l(x) - \hat{\Phi}_{h,w}^l(x'))\|_2^2, \quad (3.21)$$

其中  $w_l$  为各层学习权重。本文使用 AlexNet 作为  $\Phi$ ，数值越大表示感知差异越大。

**CLIP Score**<sup>[21]</sup>。利用 CLIP 模型的图像编码器  $E_{\text{img}}$  和文本编码器  $E_{\text{txt}}$ ，对生成样本  $x'$  与通用提示词  $p$  计算图文一致性：

$$\text{CLIP}(x', p) = \cos(E_{\text{img}}(x'), E_{\text{txt}}(p)) = \frac{E_{\text{img}}(x')^\top E_{\text{txt}}(p)}{\|E_{\text{img}}(x')\| \cdot \|E_{\text{txt}}(p)\|}. \quad (3.22)$$

**FID (Fréchet Inception Distance)**<sup>[21]</sup>。设干净样本和对抗样本的 Inception 特征分别服从  $\mathcal{N}(\mu_r, \Sigma_r)$  和  $\mathcal{N}(\mu_g, \Sigma_g)$ ，则

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\right). \quad (3.23)$$

本文实现使用 feature=64 维 Inception 特征，仅用于同一实验内部的相对比较。

### 3.3.1 防御有效性判定

在无限制攻击场景下，防御方法需要同时满足两个条件：恢复对抗样本的正确分类 (RA)，以及不过多损害干净样本性能。本文定义防御  $D_\psi$  相对于攻击方法  $A$  的净鲁棒收益为

$$\text{NRG}(D_\psi, A) = \text{RA}(D_\psi) - \Delta_{\text{clean}}(D_\psi). \quad (3.24)$$

直觉上， $\text{NRG} > 0$  意味着防御恢复的对抗鲁棒准确率超过了其对干净输入造成的准确率损失。进一步，考虑防御的计算代价后，定义代价调整鲁棒性为



$$\text{CAR}(D_\psi, A) = \frac{\text{NRG}(D_\psi, A)}{\log(2 + T_{D_\psi})}, \quad (3.25)$$

其中  $T_{D_\psi}$  为防御处理延迟（秒）。分母采用  $\log(2 + T_{D_\psi})$  是为了避免  $T < 1\text{s}$  的轻量防御因  $\log(1 + T)$  过小而被异常放大。该指标将鲁棒性恢复与计算成本联合考量：对于接近零延迟的轻量防御，分母接近  $\log 2 \approx 0.7$ ，NRG 基本保留；对于扩散净化等高延迟防御（ $T \approx 138\text{s}$ ），分母约  $\log 140 \approx 4.9$ ，即使 NRG 为正，CAR 也会被明显衰减。

资源代价被表示为向量

$$\mathbf{c} = (\bar{q}, T, M_{\text{gpu}}, M_{\text{cpu}}), \quad (3.26)$$

其中  $\bar{q}$  为平均查询次数， $T$  为处理延迟， $M_{\text{gpu}}$  和  $M_{\text{cpu}}$  分别为 GPU 显存和 CPU RSS 峰值。该代价向量使扩散攻击和扩散净化的计算负担能够与攻击效果一起被分析。

图 3-3 以四象限视角总结了上述多维评估协议的结构。

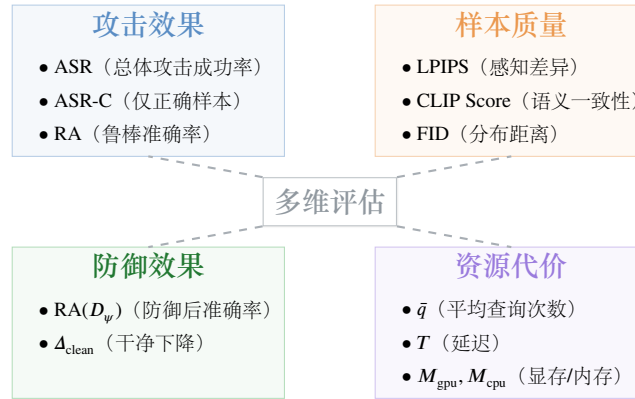


图 3-3 多维评估协议的四象限结构

### 3.3.2 多种子聚合协议

为降低随机种子对评估结果的影响，本文对每组实验使用  $R$  个随机种子重复运行，并按照算法 2 对结果进行聚合。该协议保证论文中报告的所有指标均附有标准差，读者可以据此判断结果的稳定性。

**Algorithm 2** 多种子聚合协议**Require:** 评估配置  $\mathcal{C}$ , 种子集合  $\{r_1, \dots, r_R\}$ , 指标函数集合  $\{m_1, \dots, m_J\}$ **Ensure:** 聚合指标  $\bar{m}_j \pm \sigma_j$ , 逐种子原始记录

```

1: for  $r = r_1$  to  $r_R$  do
2:   设置全局随机状态  $\text{SETSEED}(r)$ 
3:   执行攻击  $X_{\text{adv}}^{(r)}, S^{(r)}, Q^{(r)} \leftarrow \text{DIFFSEARCH}(\mathcal{C}, r)$ 
4:   for all 防御  $D_\psi \in \mathcal{C}.\text{defenses}$  do
5:     计算防御后样本  $X_{\text{def}}^{(r)} \leftarrow D_\psi(X_{\text{adv}}^{(r)})$ 
6:   end for
7:   for all 指标  $m_j$  do
8:     记录  $v_j^{(r)} \leftarrow m_j(X, X_{\text{adv}}^{(r)}, X_{\text{def}}^{(r)}, Y)$ 
9:   end for
10:  保存逐种子产物:  $\{X_{\text{adv}}^{(r)}, X_{\text{def}}^{(r)}, \mathbf{v}^{(r)}\}$ 
11: end for
12: for all 指标  $m_j$  do
13:    $\bar{m}_j \leftarrow \frac{1}{R} \sum_r v_j^{(r)}, \sigma_j \leftarrow \sqrt{\frac{1}{R-1} \sum_r (v_j^{(r)} - \bar{m}_j)^2}$ 
14: end for
15: return  $\{(\bar{m}_j, \sigma_j)\}_{j=1}^J$ 

```

### 3.4 方法扩展性

本文当前实现的候选生成算子是 Stable Diffusion v1.5 img2img。为了让论文能够承接后续补实验，本节把候选生成抽象为可替换模块：

$$\mathcal{G}_i = \mathcal{G}(x_i; \Omega, \xi), \quad (3.27)$$

其中  $\Omega$  表示算法特有控制参数。对于本文方法,  $\Omega = (p, s, g, N)$ ; 对于 AdvDiff 式方法,  $\Omega$  可包含目标分类器梯度和采样步条件; 对于 SCA 式方法,  $\Omega$  可包含反演噪声图、语义引导和加速采样器; 对于 AnyAttack 式多模态攻击,  $\Omega$  可包含目标图像、视觉语言模型损失和预训练攻击生成器。这样处理后, 本文实验系统不再只是固定平台, 而是一个围绕候选集合和评价协议组织的算法研究框架。

这种抽象也解释了为什么本文不重新实验仍能完成论文主线调整。已有实验已经覆盖本文当前实现的生成算子、候选预算、编辑强度、输入净化防御、跨模型和目标攻击补充分析; 新加入的前沿算法介绍则作为第 6 章未来工作的具体路线, 而不混入当前实验结论。换言之, 本文已经完成的是 DIFFSEARCH 的实证评估, 后续可补的是更强候选生成算子的替换实验。

## 第 4 章 算法实现与实验支撑系统

### 4.1 技术选型

为实现第 3 章提出的扩散式候选搜索与评估方法，本文构建了配套的实验支撑系统。技术选型遵循“成熟稳定、生态丰富、易于扩展”的原则，详细方案如表 4-1 所示。

表 4-1 算法实现与实验支撑系统的技术选型

| 组件     | 技术选型                  | 选型理由                       |
|--------|-----------------------|----------------------------|
| 编程语言   | Python 3.11           | 深度学习生态核心语言                 |
| CLI 框架 | Typer                 | 类型提示驱动，自动生成帮助文档            |
| 配置管理   | OmegaConf             | 轻量级，支持 YAML 合并与序列化         |
| 构建工具   | Hatchling             | 简洁的 Python 项目打包方案          |
| 深度学习框架 | PyTorch 2.x           | 动态计算图，调试便捷                 |
| 传统攻击   | 自实现 PyTorch 模块        | 接口统一，便于指标和元数据记录            |
| 扩散模型   | HuggingFace diffusers | AutoPipeline 支持 img2img 管线 |
| 图像变换   | PIL+torchvision       | 覆盖压缩、模糊、量化和张量转换            |
| 感知指标   | lpips (AlexNet)       | 标准 LPIPS 实现                |
| 分布指标   | torchmetrics (FID)    | Inception 特征统计标准实现         |
| 语义指标   | transformers (CLIP)   | CLIP 模型语义匹配                |
| 可视化    | matplotlib            | 通用绘图库，支持论文级输出              |
| 报告模板   | Jinja2                | 灵活的模板渲染引擎                  |
| 测试框架   | pytest                | Python 标准测试框架              |

在深度学习框架方面，系统选择 PyTorch 2.x 作为核心框架，主要考虑其动态计算图特性便于调试和开发，且拥有丰富的对抗攻击和防御算法生态。PyTorch 的自动微分机制是实现梯度攻击的基础，其灵活的张量操作接口也为各种图像变换提供了便利。在扩散模型方面，采用 HuggingFace diffusers 库作为 Stable Diffusion 的接口层，该库提供了标准化的 img2img 管线和模型管理功能，支持半精度推理和注意力优化等加速特性。在配置管理方面，选择 OmegaConf 作为 YAML 解析和管理工具，其支持配置合并、类型检查和环境变量插值等高级特性，使得实验配置的管理既灵活又安全。

## 4.2 模型库实现

模型库模块的核心实现包括分类器加载和扩散模型加载两部分。

分类器加载通过注册表模式管理多种模型架构。注册表维护了一个从字符串名称到工厂函数的映射字典，工厂函数负责实例化对应的模型并加载预训练权重。为了提高可复现性，系统没有直接依赖 `torchvision` 构造函数的隐式下载逻辑，而是通过可配置的权重镜像地址加载 ResNet-50<sup>[2]</sup>、ViT-B/16<sup>[2]</sup> 和 DenseNet-121<sup>[2]</sup> 的官方权重，并对 DenseNet 旧版权重键名进行兼容性映射。ImageNet-NormalizeWrapper 作为分类器的统一封装层，在前向传播中自动对  $[0, 1]$  范围的输入进行 ImageNet 标准化处理；对于 ViT-B/16，该封装层还负责将输入重采样到模型要求的  $224 \times 224$  尺寸，使下游攻击和防御模块无需关心不同分类器的预处理差异。

扩散模型加载模块使用 `diffusers` 的 `AutoPipelineForImage2Image` 从 HuggingFace Hub 或本地缓存加载预训练模型。CUDA 环境下默认以半精度加载，CPU 环境下会自动切换为 `float32` 以避免半精度数值问题。系统默认关闭 `diffusers` 安全检查器，因为实验对象来自受控研究数据集和测试管线，同时开启 `attention slicing` 以降低显存峰值。进程级缓存通过 `(model_id, device, dtype)` 三元组索引，保证同一 `worker` 内的扩散攻击和扩散净化可以复用已加载管线；批量并行时，不同 `worker` 进程各自持有独立缓存，因此 CLI 在并行运行前提供预热步骤来提前下载分类器、Stable Diffusion、LPIPS、FID 和 CLIP 相关权重。

## 4.3 对抗样本生成模块

### 4.3.1 单步梯度扰动生成

单步梯度扰动生成采用 FGSM 基线。实现流程为：(1) 将输入张量设置为需要梯度计算 (`requires_grad=True`)；(2) 前向传播计算分类损失；(3) 反向传播获取输入梯度；(4) 取梯度符号乘以扰动幅度  $\epsilon$ ，添加到原始输入上；(5) 将结果裁剪至  $[0, 1]$  的合法像素值范围。该方法每个样本只需一次前向传播和一次反向传播，计算效率高，适合作为传统扰动基线。

### 4.3.2 迭代投影梯度优化

迭代投影梯度优化采用 PGD 基线。实现要点包括：(1) 支持随机初始化（在  $[-\epsilon, \epsilon]$  范围内均匀采样初始扰动）以增强样本多样性；(2) 每步迭代中计算梯度符号、沿梯度方向移动步长  $\alpha$ ；(3) 将累积扰动投影回  $L_\infty$  范数  $\epsilon$ -球内；(4) 将最终的对抗样本裁剪至  $[0, 1]$  范围。默认配置为 20 步迭代、 $\epsilon = 0.03$ 、 $\alpha = 0.007$ 。

### 4.3.3 生成器基线方法

当前 AdvGAN 实现定位为生成器攻击接口的工程基线，而不是离线充分训练的完整 AdvGAN 复现。实现中首先构造一个四层卷积生成器，输出经  $\tanh$  约束的扰动；随后在每个输入批次上即时训练该生成器。非目标设置下，代理目标为最大化真实标签交叉熵，使生成器推动样本离开原始判别边界；若配置了 `target_class`，则改为最小化目标类别交叉熵。训练结束后，系统将生成扰动裁剪到  $[-\epsilon, \epsilon]$  区间，再与原图相加并裁剪到  $[0, 1]$  合法像素范围。该实现的查询次数按训练轮数 `epochs` 记录，元数据保存扰动半径、训练轮数、学习率、目标类别、代理目标和耗时。由于生成器并未跨批次或跨数据集累积训练，本文在实验章节将其解释为轻量生成器基线，而不是代表完整离线 AdvGAN 的最优性能。

### 4.3.4 扩散式候选搜索生成

扩散式生成模块基于 Stable Diffusion `img2img` 管线实现。系统将输入张量按 `sd_batch_size` 切分为若干子批次，转换为 PIL 图像后交给 `diffusers` 管线，在固定提示词 `a high quality photo`、变换强度 `strength` 和引导系数 `guidance_scale` 的控制下生成候选样本，再使用目标分类器判断候选是否攻击成功。与传统梯度扰动不同，扩散候选样本不受  $L_p$  范数球约束，其核心参数控制的是图像编辑强度、批量生成粒度和候选搜索预算。需要说明的是，本文实现受 AdvDiffuser 等扩散式无限制样本生成工作的启发，但并未复现其梯度注入、CAM 引导或 adversarial inpainting 机制，而是将 Stable Diffusion `img2img` 候选搜索实现为 DIFFSEARCH 的候选生成算子。为支持无 GPU 或无模型权重的单元测试，生成模块还提供 `mock` 后端，用颜色扰动、噪声和模糊模拟生成式变换；论文主要评估均使用 `sd` 后端。

候选样本筛选策略如下。对于每个输入样本，方法最多生成  $N$  轮候选，默认 `num_candidates=5`。这一默认值来自候选数  $\{1, 3, 5, 10\}$  的敏感性分析：当前数据中较小候选数已经可以达到接近饱和的 ASR，但  $N = 5$  能在保持 100% 攻击成功率的同时提供更高的 LPIPS 候选多样性，而查询开销仍明显低于  $N = 10$ 。因此本文将其作为默认评估配置，而不是声称其在所有模型和数据集上都是最优值。

候选选择采用“先成功、再择优”的规则。算法逐轮生成候选并调用分类器预测；在非目标攻击中，若候选预测类别不同于原始预测类别，则认为该候选成功；在目标攻击中，若候选预测为指定目标类别，则认为成功。对于同一样本的多个成功候选，算法选择与原图 LPIPS 距离最大的样本作为最终对抗样本，以保留更充分的生成式变化。为了降低无效计算，若一个批次中所有样本都已经

获得至少一个成功候选，则立即早停，不再生成剩余轮次。

代码还实现了失败回退机制。若样本在主循环的  $N$  轮候选中均失败，算法会只对失败子批次追加回退生成，将 strength 提高 fallback\_strength\_delta=0.20，并最多额外尝试 3 个候选。回退阶段同样遵循早停和 LPIPS 择优规则，并在攻击结果元数据中记录 fallback\_used、候选数、批大小和累计耗时。该机制使论文中的 DIFFSEARCH 实现细节与报告中的查询次数统计保持一致。

## 4.4 输入变换与净化模块

本节将第 3 章中定义的防御函数  $D_\psi$  形式化为具体的输入变换算子，并给出实现细节。

### 4.4.1 轻量级输入变换方法

三种轻量防御可以统一表示为确定性像素级变换  $D_\psi : [0, 1]^{C \times H \times W} \rightarrow [0, 1]^{C \times H \times W}$ ：

**JPEG 压缩。**设质量因子为  $q \in \{1, \dots, 100\}$ ，JPEG 防御定义为编码-解码循环：

$$D_{\text{JPEG}}(x; q) = \text{DECODE}_{\text{JPEG}}(\text{ENCODE}_{\text{JPEG}}(x, q)), \quad (4.1)$$

其中编码过程包含  $8 \times 8$  分块的离散余弦变换 (DCT) 和依赖  $q$  的量化矩阵除法取整。 $q$  越小，量化越粗，高频信息（含对抗扰动）丢失越多，但图像保真度也越低。

**高斯模糊。**设核大小为  $k$ ，标准差为  $\sigma_b$ ，高斯模糊防御定义为二维卷积：

$$D_{\text{Blur}}(x; k, \sigma_b) = x * \mathcal{K}(k, \sigma_b), \quad \mathcal{K}(i, j) = \frac{1}{2\pi\sigma_b^2} \exp\left(-\frac{i^2 + j^2}{2\sigma_b^2}\right). \quad (4.2)$$

**位深度缩减。**设目标位深为  $b$ ，量化防御定义为均匀量化：

$$D_{\text{Bit}}(x; b) = \frac{\lfloor x \cdot (2^b - 1) + 0.5 \rfloor}{2^b - 1}. \quad (4.3)$$

上述三类变换的共同特征是计算成本极低（延迟  $< 2s$ ），但本质上只能破坏像素级高频分量，对语义级编辑变换的恢复能力有限。

### 4.4.2 扩散式输入净化方法

扩散净化防御受 DiffPure<sup>[2]</sup> 的思想启发，但当前实现并不是原论文 DDPM 净化过程的完整复现，而是一个 Stable Diffusion img2img 近似净化算子。设噪声

水平参数为  $\sigma$ ，去噪步数为  $T_d$ ，实现中的净化过程可表示为：

$$\begin{aligned}\tilde{x} &= \text{clip}(x + \sigma\epsilon, 0, 1), \\ D_{\text{Purify}}(x; \sigma, T_d) &= \text{SDIMG2IMG}_{\theta}(\tilde{x}, p_{\text{null}}, s_d, g_d, T_d), \\ \epsilon &\sim \mathcal{N}(0, \mathbf{I}),\end{aligned}\tag{4.4}$$

其中  $p_{\text{null}}$  为通用提示词， $g_d$  为净化引导系数， $s_d$  为 `img2img` 去噪强度。对比式 (3.7) 可以看出，攻击和净化共享同一类扩散编辑管线，区别在于：攻击使用编辑强度  $s$  控制加噪深度以产生语义偏移，净化先用像素级高斯噪声覆盖输入中的细粒度信号，再依赖 `img2img` 去噪恢复图像内容。由于该近似净化同样会执行生成式编辑，它可能同时改变干净语义和对抗语义；这也是实验中扩散净化面对扩散攻击效果有限且干净准确率下降较大的一个实现层面原因。

实现中将 `steps` 传递给 `diffusers` 的推理步数，`img2img` 强度由  $s_d = \max(0.3, \min(0.9, 1/T_d))$  给出。因此，当 `steps` 为 10、20 或 50 时， $s_d$  均被下界裁剪为 0.3；相应消融主要改变去噪推理步数，而不是改变 `img2img` 编辑强度。防御模块同样支持 `sd_batch_size` 子批处理以控制显存峰值。

## 4.5 攻防评估流水线

第 3 章定义的评估协议在系统中实现为端到端的攻防评估流水线。算法 3 给出其伪代码，覆盖从配置加载到产物归档的完整流程。



**Algorithm 3** 单配置攻防评估流水线**Require:** YAML 配置  $\mathcal{C}$ **Ensure:** 指标文件、样本产物、配置快照

- 1: 解析  $\mathcal{C}$ : 提取模型  $f_\theta$ 、攻击方法  $A$ 、防御集合  $\{D_1, \dots, D_L\}$ 、种子集合  $\{r_1, \dots, r_R\}$
- 2: 加载数据集  $(X, Y) \leftarrow \text{LOADDATA}(\mathcal{C}.\text{dataset})$
- 3: 加载分类器  $f_\theta \leftarrow \text{LOADMODEL}(\mathcal{C}.\text{model})$
- 4: 启动资源追踪器  $\text{RESOURCETRACKER.START}()$
- 5: **for**  $r = r_1$  **to**  $r_R$  **do**
- 6:      $\text{SETSEED}(r)$
- 7:      $X_{\text{adv}}^{(r)}, \mathcal{S}^{(r)}, Q^{(r)} \leftarrow A(X, Y, f_\theta)$      ▷ 执行攻击 (Algorithm 1或基线)
- 8:     计算攻击指标:  $\text{ASR}^{(r)}, \text{ASR-C}^{(r)}$
- 9:     计算质量指标:  $\text{LPIPS}^{(r)}, \text{CLIP}^{(r)}, \text{FID}^{(r)}$
- 10:    **for all** 防御  $D_l$  **do**
- 11:        $X_{\text{def}}^{(r,l)} \leftarrow D_l(X_{\text{adv}}^{(r)}), X_{\text{clean\_def}}^{(r,l)} \leftarrow D_l(X)$
- 12:       计算  $\text{RA}^{(r,l)}, \Delta_{\text{clean}}^{(r,l)}, T_{D_l}^{(r)}$
- 13:       计算  $\text{NRG}^{(r,l)} \leftarrow \text{RA}^{(r,l)} - \Delta_{\text{clean}}^{(r,l)}$
- 14:    **end for**
- 15: **end for**
- 16: 按算法 2聚合所有指标
- 17: 停止资源追踪, 记录  $\mathbf{c} = (\bar{q}, T, M_{\text{gpu}}, M_{\text{cpu}})$
- 18: 保存产物: 配置快照、JSON/CSV 指标、样本图像
- 19: **return** 聚合指标与产物路径

## 4.6 指标计算模块

评估模块分为攻击指标、防御指标、质量指标和资源指标四类。攻击阶段首先重新计算干净样本预测和对抗样本预测, 由此得到干净准确率、对抗准确率、预测变化率、总体 ASR 和仅干净正确样本上的 ASR。对于设置了 `target_class` 的攻击, 平台额外记录目标类别命中率; 对于非目标攻击, 则主要使用预测变化和真实标签错误来刻画攻击效果。防御阶段同时评估防御后的对抗样本和防御后的干净样本, 因此能够区分鲁棒准确率提升和干净准确率损失。

LPIPS 指标使用 `lpips` 库提供的预训练 AlexNet 模型作为特征提取器。该指标在 AlexNet 的多个卷积层上提取干净样本和对抗样本的特征表示, 计算归一化的特征距离并加权求和, 得到的数值越大表示两个样本之间的感知差异越大。实现中通过 `AIGC_LPIPS_BATCH_SIZE` 环境变量控制分块大小, 避免高分辨



率样本一次性进入 LPIPS 模型造成显存峰值过高。

FID 指标使用 `torchmetrics` 提供的 `FrechetInceptionDistance` 实现,但为了降低高分辨率多实验评估的显存和时间开销,当前代码设置 `feature=64`,因此本文报告的 FID 是实验内部使用的近似分布距离指标,而不是可与外部论文标准 `feature=2048` FID 直接横向比较的数值。该指标计算干净图像集合和对抗图像集合在特征空间中的多元高斯分布参数(均值和协方差),然后计算两个分布之间的 Fréchet 距离;在同一实现和同一数据设置下,FID 值越小表示两个分布越接近。

CLIP Score 使用 `transformers` 库加载 `clip-vit-base-patch16`,分别提取图像和文本提示词的嵌入向量,计算逐样本图文嵌入余弦相似度后取平均。当前实现使用攻击配置中的通用提示词(如 `a photo` 或 `a high quality photo`)进行计算,因此该指标只反映对抗样本与通用图文提示的一致程度,不能单独证明对抗样本保持了原始类别语义。资源指标由 `ResourceTracker` 在实验运行期间每 0.5 秒采样一次,记录 CPU RSS 峰值、CUDA 峰值分配显存、CUDA 峰值保留显存,以及在 `pynvml` 可用时记录 GPU 平均利用率和峰值利用率。

## 4.7 配置化运行机制

实验支撑系统通过 `Typer` 框架实现命令行接口,提供以下核心命令:

| 命令                     | 功能说明                                   |
|------------------------|----------------------------------------|
| <code>run</code>       | 执行单个实验配置,接受 YAML 配置文件路径作为参数。           |
| <code>run-batch</code> | 批量执行多个实验配置,接受配置文件目录路径,支持顺序模式和显存感知并行模式。 |
| <code>ui</code>        | 启动基于 Gradio 的 Web UI 界面。               |
| <code>prewarm</code>   | 扫描配置目录并预下载分类器、扩散管线和质量指标模型权重。           |

批量运行命令是当前工程版本中变化较大的部分。顺序模式在当前进程中逐个执行配置,适合复用同一进程内的 `Stable Diffusion` 缓存;并行模式使用 `spawn` 上下文创建 `worker` 进程,先根据配置是否使用真实 SD 后端将任务划分为 `SD-heavy` 和 `Light` 两阶段,再按 GPU 编号创建独立进程池。多 GPU 模式下,每个 GPU 拥有独立进程池,`worker` 在导入 `PyTorch` 之前通过 `CUDA_VISIBLE_DEVICES` 绑定到指定 GPU,避免进程复用导致 GPU 亲和性失效。若某个配置因 `CUDA OOM` 失败,平台会在阶段结束后用独占 `worker` 进行串行重试。`CLI` 还实现了 `SIGINT/SIGTERM` 处理,第一次中断会关闭所有进程池并清理残留 `worker`,第二次中断则强制退出。

实验的可复现性通过以下机制保障：(1) 每个随机种子进入攻击-防御流程前统一设置 **Python** 内置随机数生成器、**NumPy** 随机数生成器和 **PyTorch** 随机数生成器；(2) 将完整的实验配置保存为 `config.yaml` 快照，记录所有影响实验结果的参数和设置；(3) 保存 `per_seed_metrics.json`、`metrics.json`、`metrics.csv`、`structured_metrics.json` 和 `metrics_wide.csv`，同时保留逐种子原始指标和跨种子聚合指标；(4) 干净样本、第一随机种子的对抗样本和防御后样本分别保存为图像文件，支持后续的定性分析和可视化检查；(5) 资源追踪器将运行时 **CPU/GPU** 开销写入同一指标文件，使论文中的效率分析可以回溯到具体实验目录。

此外，系统通过 **Python** 标准库的 **logging** 模块在运行时记录关键事件和异常信息；当前版本主要将这些信息输出到控制台或调用环境，而不是默认写入每个实验目录的独立日志文件。因此，论文中关于实验审计的依据主要是配置快照、指标文件、样本产物和批量运行摘要，而非单独的日志归档。

## 第 5 章 实验与分析

本章基于已有报告数据，对 **DIFFSEARCH** 的生成特征、防御响应和参数影响进行分析。为提高结果稳定性，本章统一采用多随机种子重复实验，并在表格中报告均值和标准差。除特别说明外，传统扰动和轻量输入变换的参数分析使用 500 张 ImageNet 子集图像，扩散候选搜索、扩散净化和跨模型分析使用 200 张 ImageNet 子集图像，随机种子均为 42、123、456。工程系统在本章中只承担配置复现和指标归档角色，实验解释围绕候选搜索方法本身展开。

### 5.1 实验设置

#### 5.1.1 代码与配置组织

实验配置已按分析目的重构为三组：论文主线目录 `configs/paper/`，机制与参数分析目录 `configs/ablations/`，以及冒烟测试目录 `configs/smoke/`。论文主线围绕传统扰动基线、**DIFFSEARCH** 候选搜索、ResNet-50 攻防矩阵、跨模型评估和目标设定边界展开；机制与参数分析进一步拆分为扰动预算、候选预算、扩散编辑强度、输入变换参数和模型结构差异等维度。每个配置文件都显式记录样本量、随机种子、模型、方法、防御、指标和报告输出目录，运行结束后会在报告目录保存解析后的 `config.yaml`，保证论文结果可以回溯到具体配置。

表 5-1 论文实验配置概览

| 实验类型           | 样本量       | 主要用途                                       |
|----------------|-----------|--------------------------------------------|
| 传统扰动基线         | 500       | FGSM、PGD 在 ResNet-50 上的扰动强度与轻量变换响应         |
| 候选搜索主线         | 200       | Stable Diffusion img2img 候选搜索与 JPEG/扩散净化响应 |
| ResNet-50 交互矩阵 | 200       | 三类生成方式与四种输入处理方法的交叉评估                       |
| 机制与参数分析        | 200 或 500 | 编辑强度、候选数、输入变换参数、净化步数和噪声水平分析                |
| 跨模型结构分析        | 200       | ResNet-50、ViT-B/16、DenseNet-121 的结构差异比较    |

5.1.2 评价指标定义

本文同时报告“ASR”和“ASR（仅正确样本）”，二者分母不同。ASR 使用全部评估样本作为分母，反映攻击结果中有多少样本满足攻击成功判定；ASR（仅正确样本）只在干净输入原本被目标模型正确分类的样本上计算，反映攻击对原本有效分类能力的破坏程度。因此，当干净准确率低于 100% 时，两个指标可能存在差异。防御效果以鲁棒准确率（Robust Accuracy, RA）为主，即模型在防御处理后的对抗样本上仍预测正确的比例；干净准确率下降用于衡量防御对正常输入的副作用。

资源开销由 ResourceTracker 记录。系统在实验运行期间采样 CPU RSS 峰值、CUDA 峰值显存分配、CUDA 保留显存，以及可用时的 GPU 利用率。由于扩散类实验需要加载 Stable Diffusion 管线，本文在防御代价分析中同时报告延迟、GPU 显存和 CPU 内存，而不再只讨论时间成本。

5.1.3 同类算法比较口径

表 5-2 本文实验中的同类方法比较层次

| 比较层次    | 纳入对象                                     | 在本文中的作用                                         |
|---------|------------------------------------------|-------------------------------------------------|
| 直接定量比较  | FGSM、PGD、修正后的生成器基线、DIFFSEARCH            | 使用同一 ImageNet 子集、目标模型、指标和报告格式，比较攻击成功率、质量指标和查询成本 |
| 防御响应比较  | JPEG、高斯模糊、位深度缩减、扩散净化                     | 检验不同攻击在输入变换和生成式净化下的鲁棒准确率与干净准确率副作用               |
| 机制性定性比较 | AI-GAN、AdvDiff、AdvDiffuser、SCA、AnyAttack | 对比训练需求、梯度引导、扩散反演、语义约束和多模态目标等机制差异，说明本文候选搜索的边界    |
| 复现证据比较  | 多随机种子、配置快照、JSON/CSV、样本归档、资源记录            | 保证直接实验结果可回溯，也为后续接入更强同类算法保留统一评价口径                |

表 5-2 把“同类算法比较”分成两类：能够在当前代码中直接运行的攻击用于定量比较，尚未完整复现的前沿生成式攻击用于机制性比较。这样处理可以避免把 RobustBench、CleverHans 或 ART 的工具覆盖范围误写成本文贡献，也能更清楚地回答本文方法相对于同类生成式攻击的差异：DIFFSEARCH 不依赖离线训练或扩散过程梯度注入，但提供了候选预算、早停回退、成功候选选择和多维指标记录。需要说明的是，当前实现已将生成器基线的代理损失修正为非目标交

叉熵上升；因此表 5-16 中的生成器基线数值需在重跑后替换，本文当前结论不再使用旧实现的 AdvGAN 数值支撑。

## 5.2 传统扰动方法基线分析

传统攻击基线使用 500 张 224×224 ImageNet 子集图像，在 ResNet-50 上评估 FGSM 和 PGD。表 5-3 给出了三次运行的均值和标准差。FGSM 的总体 ASR 为 40.60%，由于该攻击是确定性的单步梯度法，三个种子结果一致；PGD 的总体 ASR 为  $92.40\% \pm 0.72\%$ ，仅在干净正确样本上的 ASR 达到  $99.68\% \pm 0.14\%$ 。这表明 PGD 在当前设置下对干净正确样本具有较高攻击成功率；总体 ASR 略低于该值，是因为分母包含了干净输入中原本分类错误的样本。

表 5-3 传统攻击基线结果（ResNet-50，500 张图像，3 个随机种子）

| 攻击方法 | ASR (%)          | ASR (仅正确, %)     | 干净准确率 (%)        | 对抗准确率 (%)        | 查询次数 |
|------|------------------|------------------|------------------|------------------|------|
| FGSM | $40.60 \pm 0.00$ | $42.89 \pm 0.00$ | $83.00 \pm 0.00$ | $47.40 \pm 0.00$ | 1    |
| PGD  | $92.40 \pm 0.72$ | $99.68 \pm 0.14$ | $83.00 \pm 0.00$ | $0.27 \pm 0.12$  | 20   |

表 5-4 展示三种轻量防御在传统攻击上的表现。高斯模糊在 FGSM 和 PGD 下的 RA 分别为 56.00% 和 46.87%，干净准确率下降为 3.60%；JPEG 对 PGD 的恢复能力接近高斯模糊，但处理延迟更高；位深度缩减对 FGSM 保留一定效果，但对 PGD 的恢复效果较弱。这说明不同预处理方法对攻击类型较敏感，单一轻量防御难以稳定覆盖不同传统攻击。

表 5-4 传统攻击下的防御效果（RA/干净下降/延迟，粗体标出同组最优值）

| 攻击   | 防御          | RA (%)                             | 干净下降 (%)                          | 延迟 (s)                              |
|------|-------------|------------------------------------|-----------------------------------|-------------------------------------|
| FGSM | JPEG(75)    | $55.00 \pm 0.00$                   | $6.60 \pm 0.00$                   | $1.149 \pm 0.044$                   |
|      | 高斯模糊        | <b><math>56.00 \pm 0.00</math></b> | $3.60 \pm 0.00$                   | $0.009 \pm 0.010$                   |
|      | 位深度 (4 bit) | $49.20 \pm 0.00$                   | <b><math>1.00 \pm 0.00</math></b> | <b><math>0.002 \pm 0.001</math></b> |
| PGD  | JPEG(75)    | $45.13 \pm 0.31$                   | $6.60 \pm 0.00$                   | $1.124 \pm 0.013$                   |
|      | 高斯模糊        | <b><math>46.87 \pm 0.58</math></b> | $3.60 \pm 0.00$                   | $0.003 \pm 0.000$                   |
|      | 位深度 (4 bit) | $0.60 \pm 0.20$                    | <b><math>1.00 \pm 0.00</math></b> | <b><math>0.001 \pm 0.000</math></b> |

## 5.3 DIFFSEARCH 候选搜索效果分析

DIFFSEARCH 主线使用 200 张 512×512 图像，Stable Diffusion v1.5 作为 img2img 后端，默认参数为 strength=0.7、guidance\_scale=7.5、num\_

candidates=5。表 5-5 表明，该候选搜索流程三次运行均达到 100% ASR，平均查询次数为 5。LPIPS 为  $0.6242 \pm 0.0013$ ，FID 为  $3.1275 \pm 0.6512$ ，反映出生成样本相对原图存在可见的编辑变化，而不是传统的微小像素扰动。

表 5-5 扩散式生成攻击主线结果 (ResNet-50, 200 张图像, 3 个随机种子)

| 攻击方法 | ASR (%)           | ASR (仅正确, %)      | 查询均值 |
|------|-------------------|-------------------|------|
| 扩散攻击 | $100.00 \pm 0.00$ | $100.00 \pm 0.00$ | 5.00 |

| 攻击方法 | LPIPS               | CLIP Score          | FID                 |
|------|---------------------|---------------------|---------------------|
| 扩散攻击 | $0.6242 \pm 0.0013$ | $0.2046 \pm 0.0011$ | $3.1275 \pm 0.6512$ |

表 5-6 显示，JPEG 与扩散净化面对扩散攻击时效果有限。JPEG 的 RA 为  $0.67\% \pm 0.58\%$ ，扩散净化同样只有  $0.67\% \pm 0.29\%$ 。扩散净化还造成  $57.50\% \pm 2.00\%$  的干净准确率下降，并引入 121.58 秒左右的处理延迟。该节所有表格和正文均以 reports/ 下的多种子结果为准。

表 5-6 扩散攻击下的防御效果 (ResNet-50, 200 张图像)

| 防御方法        | RA (%)          | 干净下降 (%)         | 延迟 (s)              |
|-------------|-----------------|------------------|---------------------|
| JPEG(75)    | $0.67 \pm 0.58$ | $1.00 \pm 0.00$  | $1.619 \pm 0.154$   |
| 扩散净化 (50 步) | $0.67 \pm 0.29$ | $57.50 \pm 2.00$ | $121.577 \pm 0.219$ |

## 5.4 与同类攻击方法的比较

表 5-7 将本文方法放回同类攻击方法序列中比较。FGSM 和 PGD 代表传统  $L_p$  扰动基线，生成器基线代表无需调用扩散模型的学习式扰动生成接口，DIFF-SEARCH 则代表基于 Stable Diffusion img2img 的无限制候选搜索。该比较的重点不是单独追求最高 ASR，而是同时观察攻击约束、是否需要训练或采样、查询预算以及感知变化。PGD 在当前设置下已经接近饱和，但其 LPIPS 明显低于扩散候选搜索；扩散攻击同样达到 100% ASR，同时表现出更大的视觉编辑幅度，因此后续防御分析需要把它与传统扰动分开讨论。

从机制上看，DIFFSEARCH 与 AI-GAN、AdvDiff、AdvDiffuser、SCA 和 Any-Attack 等生成式攻击属于同一研究方向，但当前实现选择了更可复现的候选搜索路线。AI-GAN 类方法需要离线训练生成器，优势是推理速度快，但模型迁移和训练数据覆盖会影响攻击效果；AdvDiff 和 AdvDiffuser 通过扩散采样过程中的对抗引导增强目标性，攻击能力更强，但实现复杂度和计算开销也更高；SCA 进

表 5-7 直接可运行攻击方法的比较（ResNet-50，200 张图像；生成器基线待重跑）

| 方法         | 主要机制            | 约束/搜索空间         | ASR (%)           | LPIPS  | 查询或训练成本  |
|------------|-----------------|-----------------|-------------------|--------|----------|
| FGSM       | 单步梯度扰动          | $L_\infty$ 范数球  | $50.50 \pm 0.00$  | 0.2429 | 1 次梯度    |
| PGD        | 多步投影梯度          | $L_\infty$ 范数球  | $99.17 \pm 0.29$  | 0.1183 | 20 步梯度   |
| 生成器基线      | 小型 CNN 即时训练     | $L_\infty$ 扰动生成 | 待重跑               | 待重跑    | 50 轮批内训练 |
| DIFFSEARCH | SD img2img 候选搜索 | 无固定范数球          | $100.00 \pm 0.00$ | 0.5746 | 5 个候选预算  |

一步引入扩散反演和语义一致性引导，用于缓解无限制攻击中的语义漂移；Any-Attack 则把攻击对象扩展到视觉语言模型。本文方法没有复现这些更强生成算子的全部机制，因此其贡献应表述为“可复现的扩散候选搜索与多维评估协议”，而不是声称在算法强度上全面超过上述工作。

## 5.5 攻防交互与鲁棒性矩阵分析

为了在同一图像尺寸和同一指标集合下比较攻击特征，完整 ResNet-50 实验使用 200 张 512×512 图像，同时运行 FGSM、PGD 和 Diffusion，并对四种防御进行交叉评估。图 5-1 与表 5-8 展示攻击侧结果。PGD 与 Diffusion 都能使对抗准确率降至 0%，但二者的样本质量特征显著不同：PGD 的 LPIPS 仅为 0.1183，而 Diffusion 的 LPIPS 为 0.5746，后者体现出更强的生成式图像编辑幅度。

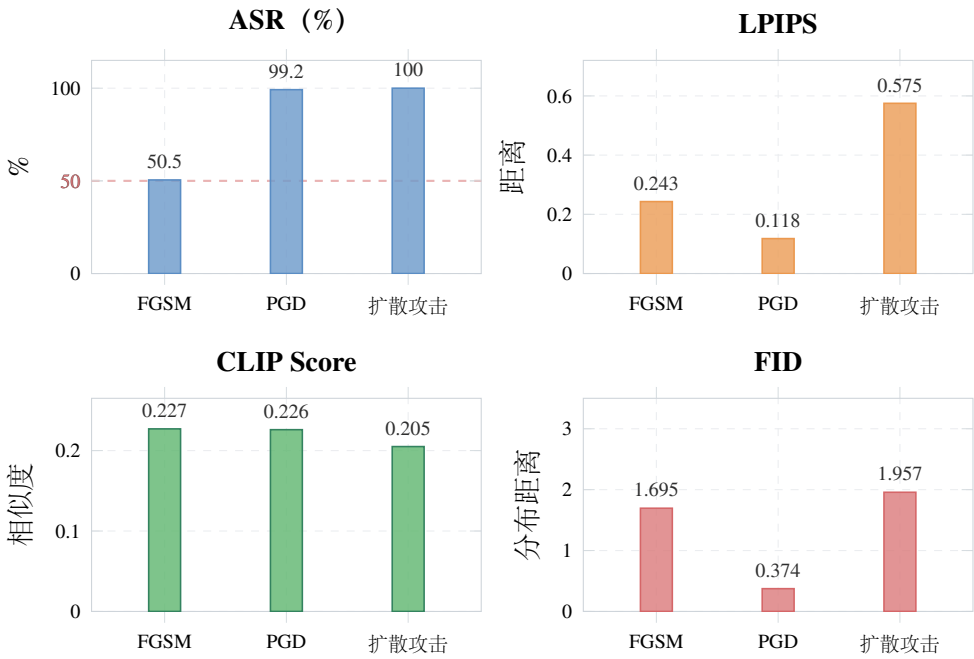


图 5-1 三种攻击方法的多维指标对比

表 5-9 和图 5-3 给出攻防交叉结果。轻量防御可以部分恢复传统攻击下的准

表 5-8 完整攻防实验中的攻击效果对比（ResNet-50，200 张图像）

| 攻击方法 | ASR (%)           | ASR（仅正确，%）        | LPIPS               | CLIP Score          | FID                 |
|------|-------------------|-------------------|---------------------|---------------------|---------------------|
| FGSM | $50.50 \pm 0.00$  | $52.44 \pm 0.00$  | $0.2429 \pm 0.0000$ | $0.2268 \pm 0.0000$ | $1.6950 \pm 0.0000$ |
| PGD  | $99.17 \pm 0.29$  | $100.00 \pm 0.00$ | $0.1183 \pm 0.0002$ | $0.2262 \pm 0.0000$ | $0.3743 \pm 0.0005$ |
| 扩散攻击 | $100.00 \pm 0.00$ | $100.00 \pm 0.00$ | $0.5746 \pm 0.0019$ | $0.2048 \pm 0.0005$ | $1.9565 \pm 0.1738$ |

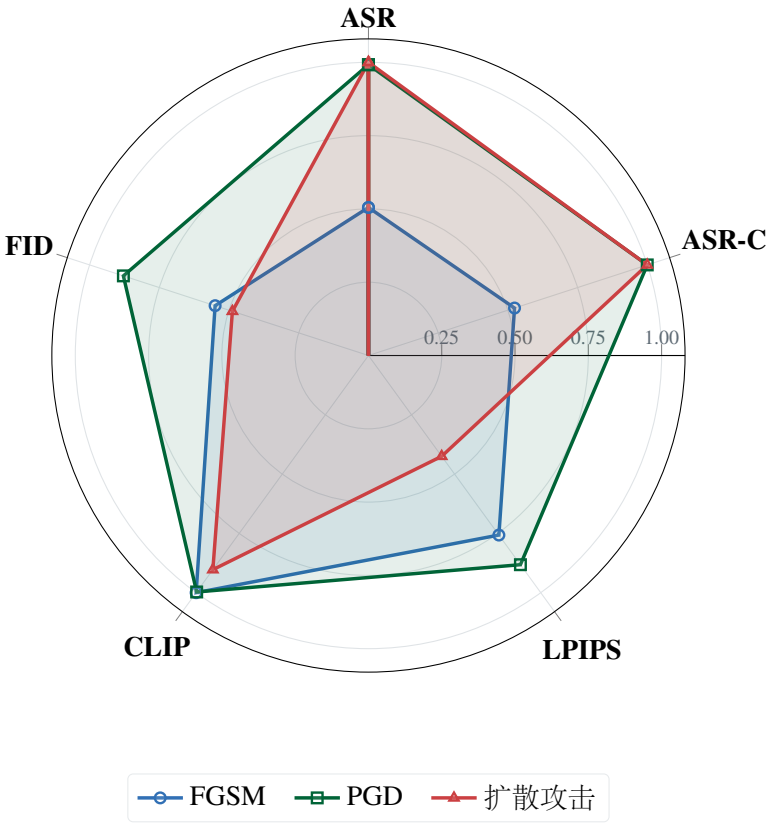


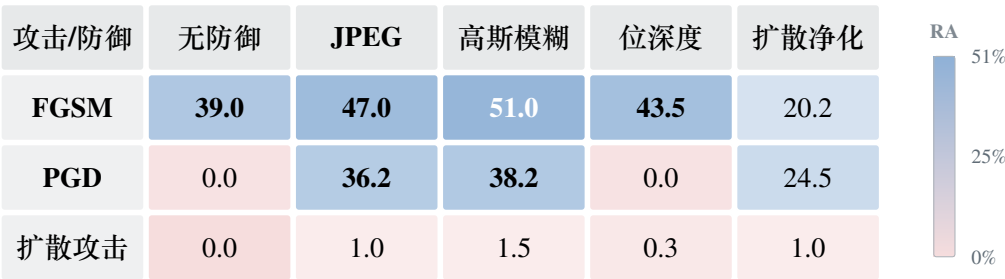
图 5-2 三种攻击方法的归一化雷达图



确率，例如高斯模糊使 FGSM 的 RA 达到 51.00%，JPEG 和高斯模糊使 PGD 的 RA 分别恢复到 36.17% 和 38.17%。然而，对扩散攻击而言，四种防御的 RA 均不超过 1.50%。这说明在当前 Stable Diffusion img2img 候选搜索生成的样本上，压缩、平滑、量化和去噪都难以恢复模型的原始判别结果。

表 5-9 完整攻防交叉评估：鲁棒准确率 RA（%，粗体标出每类攻击下最高值）

| 攻击方法 | 无防御   | JPEG         | 高斯模糊                | 位深度          | 扩散净化         |
|------|-------|--------------|---------------------|--------------|--------------|
| FGSM | 39.00 | 47.00 ± 0.00 | <b>51.00 ± 0.00</b> | 43.50 ± 0.00 | 20.17 ± 1.26 |
| PGD  | 0.00  | 36.17 ± 0.58 | <b>38.17 ± 0.58</b> | 0.00 ± 0.00  | 24.50 ± 0.50 |
| 扩散攻击 | 0.00  | 1.00 ± 0.50  | <b>1.50 ± 0.50</b>  | 0.33 ± 0.29  | 1.00 ± 0.50  |



颜色映射：蓝色越深表示鲁棒准确率越高，红色表示接近零

图 5-3 攻防交叉评估热力图：鲁棒准确率（%）

5.5.1 防御代价分析

图 5-5 重绘了防御方法的干净准确率下降和延迟。轻量防御中，高斯模糊与位深度缩减的延迟低于 0.04 秒，JPEG 延迟约 1.43 秒；扩散净化的平均延迟约 137.8 秒，比轻量防御高两个数量级。表 5-10 进一步给出资源开销。完整 ResNet-50 实验加载扩散攻击与扩散净化后，峰值 GPU 分配显存约 8.3 GB，CPU RSS 峰值约 7.34 GB；扩散攻击主线峰值 GPU 分配显存约 19.2 GB，主要来自 Stable Diffusion 后端和较高分辨率批处理。资源指标表明，扩散净化除时间代价外，也会提高部署所需的计算资源。

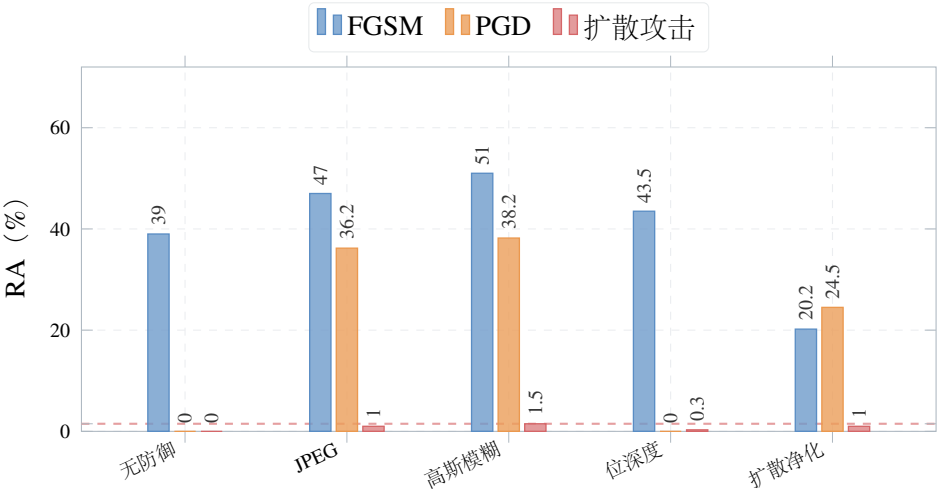


图 5-4 不同防御方法在三类攻击下的 RA 对比

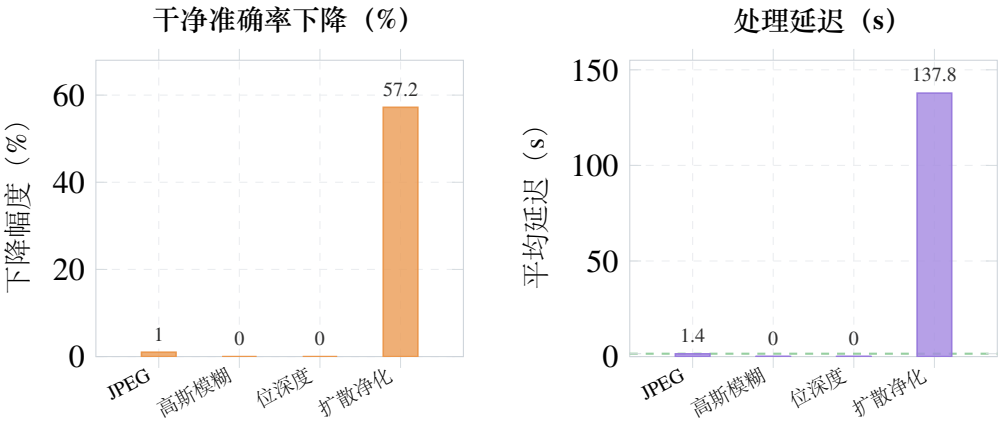


图 5-5 防御代价：干净准确率下降与处理延迟

表 5-10 代表性实验的资源占用记录

| 实验             | GPU 峰值分配显存 (MB) | GPU 保留显存 (MB) | CPU RSS 峰值 (MB) |
|----------------|-----------------|---------------|-----------------|
| 传统攻击基线         | 45337.3         | 45636.0       | 2125.0          |
| 扩散攻击主线         | 19192.2         | 21014.0       | 4599.7          |
| ResNet-50 完整矩阵 | 8302.0          | 8886.0        | 7343.8          |
| 扩散防御交叉实验       | 18644.5         | 20260.0       | 4344.3          |

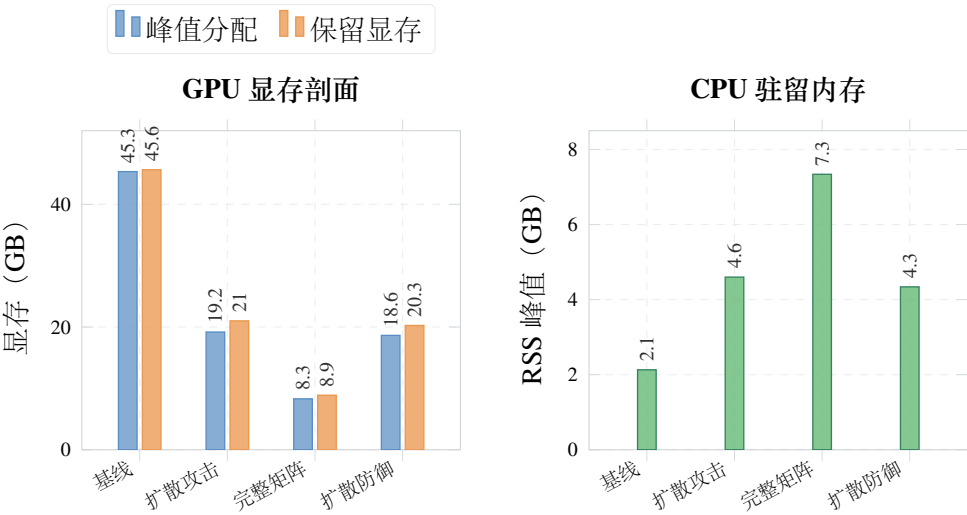


图 5-6 代表性实验的 GPU/CPU 资源占用剖面

图 5-6 进一步把表 5-10 中的资源记录转化为可比较的剖面。GPU 维度中，传统攻击基线的峰值分配显存达到 45.34 GB，显著高于扩散攻击主线和扩散防御交叉实验；这一现象可能与该实验的样本量、批大小、梯度计算和 CUDA 运行时缓存等因素共同相关，仍需要更细粒度的运行时剖析才能确认主要来源。扩散攻击主线和扩散防御交叉实验的 GPU 保留显存分别约为 21.01 GB 和 20.26 GB，说明 Stable Diffusion 后端即使在样本量较小的设置下也会持续占用较高显存池。

CPU 维度呈现另一种资源形态：ResNet-50 完整矩阵的 CPU RSS 峰值最高，达到 7.34 GB，而其 GPU 峰值分配显存是四组实验中最底的 8.30 GB。这表明完整矩阵实验的资源压力并不只来自单次模型推理，也可能与指标聚合、样本缓存、报告归档和多组攻防结果管理有关。因此，平台在资源记录中同时保留 GPU 分配显存、GPU 保留显存和 CPU RSS 是必要的；只报告单一显存峰值会掩盖实验类型之间的资源差异，也不利于后续根据硬件条件选择批大小、候选数量和报告保存策略。

## 5.6 关键机制与参数敏感性分析

### 5.6.1 扰动预算与迭代步数敏感性

除默认设置外，报告目录中还包含 FGSM 扰动幅度、PGD 扰动幅度和 PGD 迭代步数的参数分析。表 5-11 汇总了这些结果。FGSM 对抗扰动幅度较为敏感，当  $\epsilon$  从 0.01 提升到 0.15 时，总体 ASR 从 41.60% 增至 67.80%，对抗准确率从 46.40% 降至 29.20%。相比之下，PGD 在较小扰动幅度下已经接近饱和： $\epsilon = 0.01$  时仅正确样本 ASR 已达到 99.04%， $\epsilon = 0.15$  时进一步达到 100.00%。这说明在当前 ResNet-50 与 ImageNet 子集设置下，多步优化比单步扰动幅度更能解释攻击强度差异。

表 5-11 扰动预算参数敏感性分析（500 张图像，3 个随机种子，粗体标出同类设置中的最高攻击强度）

| 攻击   | 参数                | ASR (%)                            | ASR (仅正确, %)                        | 对抗准确率 (%)   | 查询均值 |
|------|-------------------|------------------------------------|-------------------------------------|-------------|------|
| FGSM | $\epsilon = 0.01$ | 41.60                              | 44.10                               | 46.40       | 1    |
|      | $\epsilon = 0.03$ | 40.60                              | 42.89                               | 47.40       | 1    |
|      | $\epsilon = 0.08$ | 48.40                              | 48.19                               | 43.00       | 1    |
|      | $\epsilon = 0.15$ | <b>67.80</b>                       | <b>65.54</b>                        | 29.20       | 1    |
| PGD  | $\epsilon = 0.01$ | 89.67 $\pm$ 0.12                   | 99.04 $\pm$ 0.24                    | 0.80        | 20   |
|      | $\epsilon = 0.03$ | 92.40 $\pm$ 0.72                   | 99.68 $\pm$ 0.14                    | 0.27        | 20   |
|      | $\epsilon = 0.08$ | 95.27 $\pm$ 0.12                   | 99.92 $\pm$ 0.14                    | 0.07        | 20   |
|      | $\epsilon = 0.15$ | <b>98.27 <math>\pm</math> 0.42</b> | <b>100.00 <math>\pm</math> 0.00</b> | <b>0.00</b> | 20   |

PGD 步数分析进一步说明了扰动强度与计算预算之间的关系。7 步 PGD 的总体 ASR 为 89.53%  $\pm$  0.61%，仅正确样本 ASR 为 96.22%  $\pm$  0.28%；20 步时分别提升至 92.40%  $\pm$  0.72% 和 99.68%  $\pm$  0.14%；40 步时仅正确样本 ASR 达到 100.00%，但总体 ASR 为 91.73%  $\pm$  0.64%，没有带来与查询次数翻倍相匹配的增益。因此本文选择 20 步作为传统扰动基线配置，它在攻击强度和计算成本之间更均衡。

### 5.6.2 扩散编辑强度敏感性

表 5-12 和图 5-7 展示 strength 参数的影响。结果显示，即使在  $s = 0.3$  时，扩散攻击总体 ASR 也已达到 91.83%  $\pm$  1.04%，说明当前图像子集和模型对生成式编辑十分敏感。随着强度提升到 0.7，ASR 达到 100%，LPIPS 也从 0.3641 上升到 0.6203。攻击强度的主要作用不再只是从低 ASR 提升到高 ASR，而是在高成功率区间内进一步增加候选样本的感知变化幅度。

表 5-12 扩散编辑强度敏感性分析

| 强度  | ASR (%)       | ASR (仅正确, %)  | LPIPS  | CLIP Score | FID    |
|-----|---------------|---------------|--------|------------|--------|
| 0.3 | 91.83 ± 1.04  | 90.65 ± 1.27  | 0.3641 | 0.1967     | 2.4090 |
| 0.5 | 99.83 ± 0.29  | 99.80 ± 0.35  | 0.4973 | 0.2003     | 4.4405 |
| 0.7 | 100.00 ± 0.00 | 100.00 ± 0.00 | 0.6203 | 0.2043     | 3.0497 |

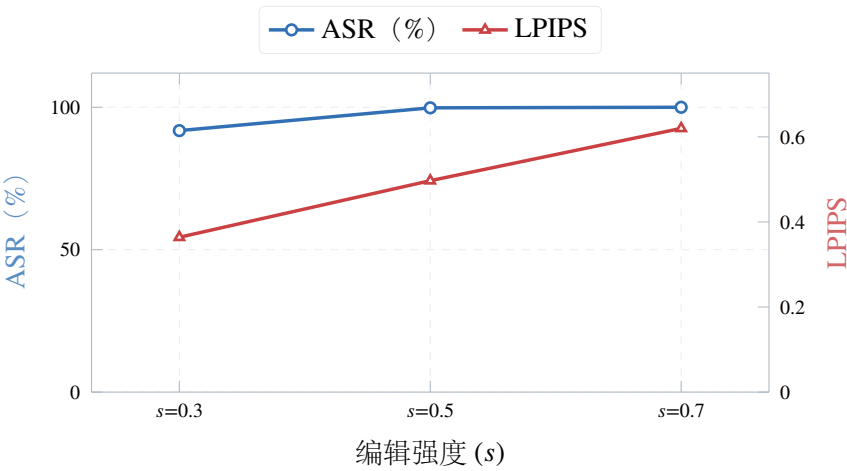


图 5-7 扩散编辑强度与 ASR/LPIPS 的关系

5.6.3 候选预算与搜索控制机制

候选预算分析用于解释 DiffSEARCH 中的默认  $N = 5$ 。表 5-13 显示， $N = 1$  已能达到 100% ASR，但平均 LPIPS 为 0.5791； $N = 5$  同样达到 100% ASR，LPIPS 提升到 0.6242； $N = 10$  的 LPIPS 最高，但平均查询次数增长到 7.34，且 ASR 未进一步提升。结合早停与失败回退机制，候选上限不等同于固定查询次数，算法会在所有样本成功后提前停止，并只对失败样本追加回退候选。

表 5-13 扩散攻击候选数量消融

| 候选上限 $N$ | ASR (%)       | 平均查询次数 | LPIPS  | 说明         |
|----------|---------------|--------|--------|------------|
| 1        | 100.00 ± 0.00 | 1.11   | 0.5791 | 依赖主候选和少量回退 |
| 3        | 99.83 ± 0.29  | 3.02   | 0.6031 | 成本适中       |
| 5        | 100.00 ± 0.00 | 5.00   | 0.6242 | 默认配置       |
| 10       | 99.83 ± 0.29  | 7.34   | 0.6278 | 多样性提升有限    |

5.6.4 扩散净化参数敏感性

扩散净化步数消融显示，增加去噪步数并不能有效防御扩散攻击。10 步和 20 步的 RA 均为 0.50%，50 步仅提升到 0.67%，但延迟从 40.95 秒增长到 121.69

秒。噪声水平消融也呈现类似现象：噪声 0.05、0.10、0.20 对应的 RA 分别为 1.33%、0.67%、1.00%，但干净准确率下降从 34.00% 扩大到 75.83%。这说明在当前设置下，扩散净化的参数调节主要改变副作用大小，未能有效恢复扩散攻击后的模型判别结果。

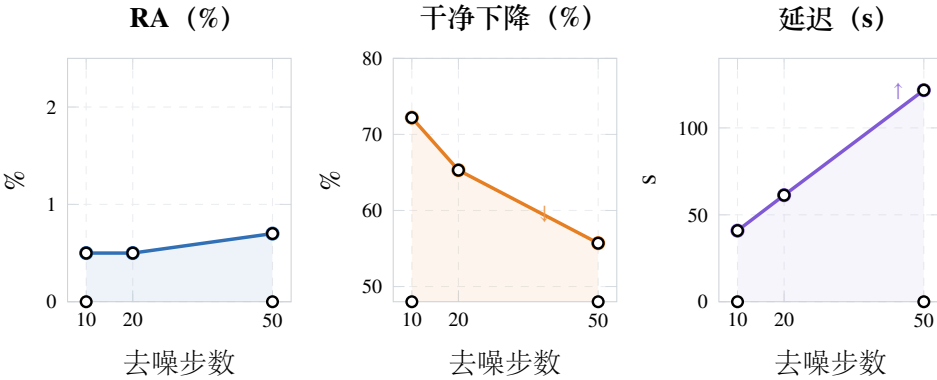


图 5-8 扩散净化步数对 RA、干净下降和延迟的影响

表 5-14 扩散净化噪声水平消融

| 噪声水平 | RA (%) | 干净下降 (%) | 延迟 (s) |
|------|--------|----------|--------|
| 0.05 | 1.33   | 34.00    | 122.28 |
| 0.10 | 0.67   | 55.67    | 121.55 |
| 0.20 | 1.00   | 75.83    | 122.45 |

5.6.5 轻量输入变换参数敏感性

轻量防御的参数扫描提供了比单一默认配置更细的解释。JPEG 质量因子越低，压缩越强，对 PGD 的恢复越明显，但对干净输入的副作用也越大。表 5-15 显示，JPEG(25) 可将 PGD 下的 RA 提升到 59.67%，但干净准确率下降 14.80%；JPEG(90) 的干净下降只有 3.80%，PGD 下 RA 却只有 11.60%。因此 JPEG 防御呈现典型的鲁棒性-保真度权衡。

高斯模糊和位深度缩减也呈现类似但形态不同的权衡。模糊核从  $3 \times 3$  增至  $7 \times 7$  时，PGD 下 RA 从 0.40% 提升到 60.13%，但干净下降也从 0.00% 增至 9.00%；继续增至  $9 \times 9$  后，RA 反而略降至 58.20%，干净下降扩大到 16.20%。位深度缩减在 2 bit 时对 FGSM 和 PGD 均能达到约 55%–58% 的 RA，但干净下降为 11.80%；当位深度提高到 4 bit 及以上时，对 PGD 的恢复效果很低。该结果说明，轻量预处理防御并不存在单调最优参数，实际部署必须同时约束干净准确率损失和攻击类型。

表 5-15 轻量防御参数消融：RA 与干净准确率下降（500 张图像）

| 防御   | 参数                    | FGSM RA (%) | PGD RA (%)       | 干净下降 (%) |
|------|-----------------------|-------------|------------------|----------|
| JPEG | quality=25            | 59.20       | 59.67 $\pm$ 1.14 | 14.80    |
| JPEG | quality=50            | 56.60       | 57.47 $\pm$ 0.42 | 9.40     |
| JPEG | quality=75            | 55.00       | 45.13 $\pm$ 0.31 | 6.60     |
| JPEG | quality=90            | 50.20       | 11.60 $\pm$ 0.20 | 3.80     |
| 高斯模糊 | $k = 3, \sigma = 0.5$ | 47.80       | 0.40 $\pm$ 0.00  | 0.00     |
| 高斯模糊 | $k = 5, \sigma = 1.0$ | 56.00       | 46.87 $\pm$ 0.58 | 3.60     |
| 高斯模糊 | $k = 7, \sigma = 1.5$ | 60.60       | 60.13 $\pm$ 0.12 | 9.00     |
| 高斯模糊 | $k = 9, \sigma = 2.0$ | 57.60       | 58.20 $\pm$ 1.22 | 16.20    |
| 位深度  | 2 bit                 | 57.80       | 55.40 $\pm$ 0.69 | 11.80    |
| 位深度  | 3 bit                 | 53.60       | 24.60 $\pm$ 0.20 | 3.80     |
| 位深度  | 4 bit                 | 49.20       | 0.60 $\pm$ 0.20  | 1.00     |
| 位深度  | 5 bit                 | 48.00       | 0.27 $\pm$ 0.12  | 0.60     |
| 位深度  | 6 bit                 | 47.80       | 0.27 $\pm$ 0.12  | -0.60    |

### 5.6.6 生成器基线修正与目标设定边界

为加强同类生成式攻击的比较，本文保留生成器基线实验，但不再使用旧实现的数值作为论文结论。旧版本在每个批次上即时训练一个小型卷积生成器，却直接最小化真实标签交叉熵，损失方向更接近提升正确分类而不是攻击目标；当前实现已将非目标生成器基线修正为最大化真实标签交叉熵，并在配置给出 `target_class` 时切换为目标类别交叉熵最小化。因此，生成器基线需要按表 5-16 重新运行后再填入最终数值。该处理既避免把错误代理损失下的低 ASR 误解为生成器路线无效，也使表 5-7 中的同类算法比较具有一致实验口径。

表 5-16 生成器基线重跑占位与目标扩散攻击补充结果

| 实验    | 参数                                   | ASR (%)         | ASR (仅正确, %)    | 对抗准确率 (%) | 查询均值 |
|-------|--------------------------------------|-----------------|-----------------|-----------|------|
| 生成器基线 | $\epsilon = 0.03, \text{epochs}=25$  | 待重跑             | 待重跑             | 待重跑       | 25   |
| 生成器基线 | $\epsilon = 0.03, \text{epochs}=100$ | 待重跑             | 待重跑             | 待重跑       | 100  |
| 生成器基线 | $\epsilon = 0.08, \text{epochs}=50$  | 待重跑             | 待重跑             | 待重跑       | 50   |
| 目标扩散  | <code>target_class=0</code>          | 0.17 $\pm$ 0.29 | 0.20 $\pm$ 0.35 | 81.83     | 7.99 |

重跑时应优先执行三组生成器参数配置和 `advgan_vs_defenses`，并同步更新表 5-7 中的生成器基线行。目标扩散实验则从另一个角度揭示了无限制



攻击的难点：在不改变提示词和搜索策略、仅指定 ImageNet 第 0 类为目标时，表 5-16 按攻击结果成功标记统计的 ASR 仅为  $0.17\% \pm 0.29\%$ ，运行器按最终预测是否等于目标类统计的目标命中率为  $0.67\% \pm 0.29\%$ 。后者可能包含原始预测已经属于目标类的样本，因此不应与非目标 ASR 直接比较；同一实验中的预测变化率为  $0.17\% \pm 0.29\%$ 。与非目标扩散攻击的 100% ASR 相比，目标攻击需要更强的类别条件引导或显式优化，而不是简单复用非目标 img2img 搜索流程。

## 5.7 跨模型分析

跨模型实验比较 ResNet-50、ViT-B/16 与 DenseNet-121。表 5-17 显示，扩散攻击在三种模型上均达到 100% ASR，说明其攻击效果在当前三类模型设置下具有一致性。传统攻击的模型差异更明显：ViT-B/16 上 FGSM ASR 为 61.00%，DenseNet-121 上 FGSM ASR 达到 94.50%，而 ResNet-50 在相同分辨率完整实验中为 50.50%。扩散净化对 ViT-B/16 干净准确率下降较小（约 28.33%），但对 DenseNet-121 下降约 45.67%，体现出净化防御对模型结构和预处理分布的敏感性。

表 5-17 跨模型攻击与扩散净化结果

| 模型           | 干净准确率 (%) | FGSM ASR (%) | PGD ASR (%) | 扩散攻击 ASR (%) |
|--------------|-----------|--------------|-------------|--------------|
| ResNet-50    | 82.00     | 50.50        | 99.50       | 100.00       |
| ViT-B/16     | 84.00     | 61.00        | 94.33       | 100.00       |
| DenseNet-121 | 72.50     | 94.50        | 98.50       | 100.00       |

图 5-9 从另一个角度展示表 5-17 的跨模型差异。横轴表示模型在干净样本上的基础准确率，纵轴表示 FGSM 的攻击成功率；气泡半径按扩散净化带来的干净准确率下降（%）线性映射到最小/最大半径以增强对比，各气泡旁直接标注该百分比。ResNet-50 位于较高干净准确率、较低 FGSM ASR 的区域，说明其在单步梯度扰动下相对更稳定，但气泡最大且旁注约 57.50%，意味着扩散净化对其干净输入副作用最大。DenseNet-121 则呈现相反形态：干净准确率较低，但 FGSM ASR 接近 95%，旁注干净下降约 45.67%，表明其在当前预处理和样本子集下对单步扰动更敏感。ViT-B/16 的气泡最小、旁注约 28.33%，说明扩散净化对其干净准确率损伤相对较轻，但其 FGSM ASR 仍高于 ResNet-50。

这一图形化结果补充了三线表难以直接呈现的结构关系：模型的干净准确率、传统攻击敏感性和净化副作用并不存在简单线性对应。扩散攻击在三个模型上均达到 100% ASR，因此它不是区分模型差异的主要维度；真正需要进一步分析的是传统梯度攻击与输入净化在不同模型结构上的交互差异。对评估协议



而言，这说明跨模型分析不能只复用同一组攻击成功率表格，还应同步记录防御副作用和资源代价，才能判断某一模型是否真的更适合实际部署。

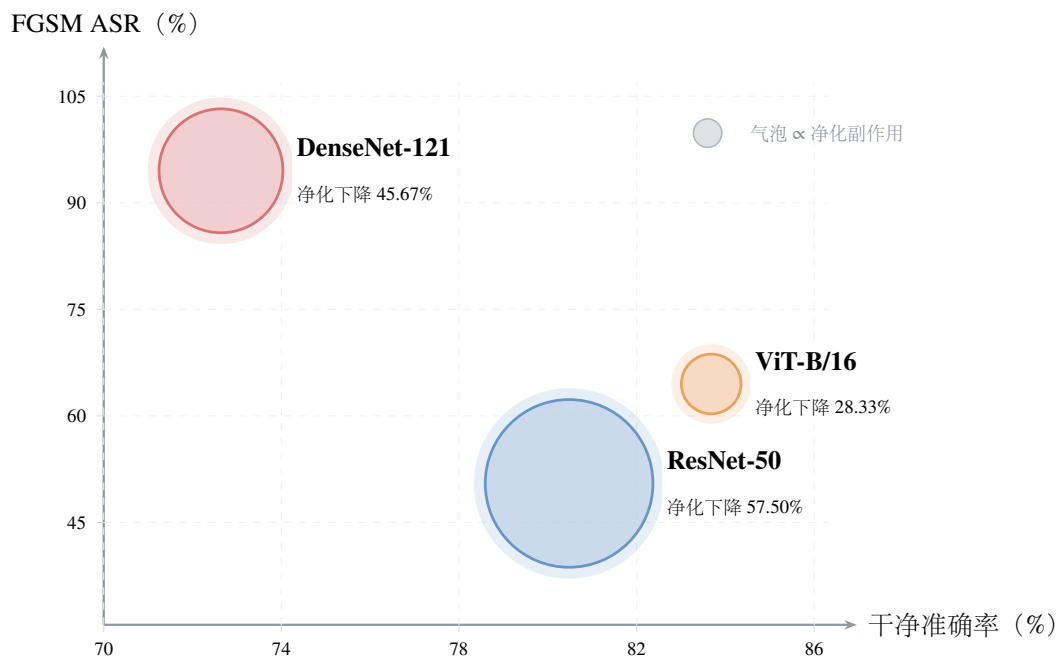


图 5-9 跨模型攻击敏感性气泡散点图（气泡半径按扩散净化后的干净准确率下降映射；旁注为对应百分比）

## 5.8 实验总结

上述实验支持三点结论。第一，扩散式候选搜索在更大样本量 and 多随机种子设置下仍保持较高成功率，且其 **LIIPS** 和对抗准确率特征不同于传统像素扰动。第二，轻量防御能在传统攻击上恢复部分准确率，但对扩散式编辑攻击作用有限；扩散净化虽然更复杂，却同样未能有效抵抗扩散攻击，并带来较高延迟、显存占用和干净准确率损失。第三，参数消融揭示了若干非单调现象：**PGD** 步数增加到一定程度后边际收益变小，候选数量主要影响感知多样性和查询预算，**JPEG**、高斯模糊和位深度缩减均存在鲁棒性与干净准确率之间的权衡，目标扩散攻击也明显难于非目标攻击。这些结果共同说明，**DIFFSEARCH** 应通过“候选搜索机制 + 多维证据”理解，而不是只用单次 **ASR** 或系统功能覆盖来概括。

## 第 6 章 总结与展望

### 6.1 工作总结

本文围绕 AIGC 场景下扩散式无限制对抗样本的生成与评估问题，提出并实现了基于 Stable Diffusion img2img 的候选搜索方法 DIFFSEARCH。与以往主要关注  $L_p$  范数扰动的攻击不同，本文方法把生成式编辑算子、候选预算、成功判定、早停回退和 LPIPS 择优组织为一个完整流程，并用 ASR、ASR-C、RA、LPIPS、CLIP Score、FID、延迟和资源占用等指标共同解释攻击结果。实验支撑系统用于保证配置、随机种子、指标和样本产物能够复现，但论文的主线已经从平台工程转向扩散攻击方法与评估协议。

在方法层面，本文形式化给出了扩散候选生成算子、非目标/目标攻击成功判定、成功候选集合、LPIPS 选择准则、失败回退强度和代价向量等定义。相关工作部分补充了 AdvDiff、AdvDiffuser、SCA、AnyAttack 等前沿路线，使本文当前实现与后续可扩展算法之间形成清晰关系：当前实验验证的是 img2img 候选搜索主线，后续可以替换候选生成算子，引入梯度引导、扩散反演、语义一致性约束或视觉语言模型目标攻击。

在实验层面，本文基于重构后的配置体系重新运行并整理了更大规模实验。传统攻击实验扩展到 500 张图像，扩散类实验扩展到 200 张图像，主要实验均使用 42、123、456 三个随机种子报告均值和标准差。实验结果表明，ResNet-50 上 PGD 对干净正确样本的 ASR 达到  $99.68\% \pm 0.14\%$ ，扩散式生成攻击在 200 张高分辨率图像上达到 100.00% ASR；轻量防御对扩散攻击的 RA 不超过 1.50%，扩散净化在该场景下同样效果有限，并带来约 121–138 秒级延迟和较高干净准确率下降。进一步的参数消融表明，传统攻击强度、防御预处理参数、候选数量和目标攻击设定都会影响实验解释，单一默认配置不足以描述完整攻防图景。修订后的实验章还把生成器基线从旧实现结果中剥离出来，作为待重跑的同类算法对照，避免用错误代理损失下的低 ASR 支撑结论。跨模型实验进一步显示，扩散攻击在 ResNet-50、ViT-B/16 和 DenseNet-121 上均达到 100% ASR，体现了生成式编辑攻击在当前模型和数据设置下的一致效果。

## 6.2 不足与展望

本文仍有若干限制。首先，当前生成式攻击主要围绕 Stable Diffusion v1.5 的 img2img 管线展开，尚未实现 AdvDiff 式梯度引导、SCA 式扩散反演或 AnyAttack 式多模态目标攻击。其次，本文采用 ImageNet 子集以控制计算成本，样本量与完整 ImageNet 验证集相比仍属于中等规模评估。第三，当前语义一致性主要依赖通用提示词 CLIP Score 和人工样例观察，尚未引入类别级文本描述、分割一致性或人类评价来严格约束无限制攻击的语义保持。

未来工作可以从三方面推进。其一，扩展候选生成算法，引入 AdvDiff 的对抗采样引导、SCA 的 DDPM 反演和语义一致性控制，以及 AnyAttack 面向视觉语言模型的目标攻击，使本文框架从分类器扩展到多模态模型。其二，增强防御侧研究，探索语义一致性检测、多模型交叉验证、生成轨迹水印和鲁棒特征学习等方法，而不仅依赖输入净化。其三，进一步完善实验支撑能力，包括分布式批量运行、实验数据库、可视化看板、与 RobustBench/ART 等生态的适配层，以及更细粒度的资源剖析。通过这些扩展，本文方法可以从当前的 img2img 候选搜索评估进一步发展为面向 AIGC 安全研究的可持续算法评测框架。

# 致 谢

衷心感谢导师在本文研究过程中的悉心指导和大力支持。从选题到实验设计，从论文写作到反复修改，导师倾注了大量的时间和精力，给予了我宝贵的学术指导和方法论启发。

感谢实验室的各位同学在技术讨论和实验过程中给予的无私帮助。在平台开发和实验调试过程中，同学们提出了许多建设性的意见和建议，使本文的研究工作得以顺利推进。

感谢复旦大学计算机科学技术学院提供的良好学习环境和计算资源支持，为本文的实验工作提供了必要的硬件保障。

感谢开源社区的贡献者们开发和维护了 **PyTorch**、**HuggingFace Diffusers**、**torchattacks** 等开源工具和框架，为本文的平台开发提供了重要的技术基础。

最后，衷心感谢我的家人一直以来的理解、支持和鼓励。正是家人的无条件支持，让我能够专注于学业和研究，顺利完成本科阶段的学习任务。

## 复旦大学 学位论文独创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。论文中除特别标注的内容外，不包含任何其他个人或机构已经发表或撰写过的研究成果。对本研究做出重要贡献的个人和集体，均已在论文中作了明确的声明并表示了谢意。本声明的法律结果由本人承担。

作者签名：\_\_\_\_\_ 日期：\_\_\_\_\_

## 复旦大学 学位论文使用授权声明

本人完全了解复旦大学有关收藏和利用博士、硕士学位论文的规定，即：学校有权收藏、使用并向国家有关部门或机构送交论文的印刷本和电子版本；允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其它复制手段保存论文。涉密学位论文在解密后遵守此规定。

作者签名：\_\_\_\_\_ 导师签名：\_\_\_\_\_ 日期：\_\_\_\_\_